

Machine Learning – Lecture Summaries

Summer Semester 2026

Tobias Laser

25. Mai 2026

Inhaltsverzeichnis

1	Lecture 1: Introduction and Elements of Probabilistic Modelling	1
1.1	Introduction: What should an AI system do for us?	1
1.2	Example: Classification	2
1.3	Different Types of Learning	3
1.4	Probabilistic Answers as Optimal Inference	4
1.5	Joint Probabilities and Conditional Probabilities	5
1.6	Sum Rule and Marginal Probabilities	6
1.7	Product Rule	7
1.8	Bayes' Rule	7
1.9	Introductory Example: Pick-One-Berry-From-Two-Bowls	9
1.10	Example Algorithms: Outlook	9
1.11	Evidence of Strong Models in Biological Vision	10
1.12	Prüfungsrelevante Kurzfassung	11
2	Lecture 2: Probabilistic Calculus	11
2.1	Overview	11
2.2	Generality of the Product Rule, Sum Rule and Bayes' Rule	12
2.3	Continuous Random Variables and Probability Density Functions	13
2.4	Product Rule, Sum Rule and Bayes' Rule for Continuous Random Variables	15
2.5	Density Estimation Using Histograms	16
2.6	Expectation Values	17
2.7	Sampling Approximation of Expectation Values	18
2.8	Mean, Variance, Covariance and Correlation	19
2.9	Important Probability Distributions	21
2.10	Prüfungsrelevante Kurzfassung	22
3	Lecture 3: Probabilities and Estimators	23
3.1	Overview	23
3.2	Parameter Estimation for Uniform Distributions	24
3.3	Heuristic Estimation Using Minimum and Maximum	25
3.4	Estimation Using the Method of Moments	26
3.5	Which Estimation Method Is Better?	28
3.6	Mean and Variance of Functions of Random Numbers	29
3.7	i.i.d. Random Variables	30
3.8	Statistical Estimators	31
3.9	Bias and Variance of Estimators	32
3.10	Bias and Variance of the Sample Mean Estimator	33
3.11	Bias of the Naive Sample Variance Estimator	35
3.12	An Unbiased Estimator for the Sample Variance	36
3.13	Maximum Likelihood Estimators	37
3.14	Maximum A-Posteriori Estimators	38

3.15	Bayesian Estimators	40
3.16	Standard Derivation Technique for Maximum Likelihood Estimators	40
3.17	Standard Derivation Technique for MAP Estimators	41
3.18	Prüfungsrelevante Kurzfassung	42
4	Lecture 4: Maximum Likelihood and Regression	44
4.1	Overview	44
4.2	Recap: Estimators	45
4.3	Recap: Maximum Likelihood Estimators	45
4.4	Uniform Distribution as Maximum Likelihood Estimation	47
4.5	Testing Maximum Likelihood Estimators for Gaussian Distributions	48
4.6	Discussion of the Maximum Likelihood Method	49
4.7	Regression: Problem Definition and Maximum Likelihood for Labeled Data	50
4.8	Examples of Regression Problems	51
4.9	Linear Regression: Definition	52
4.10	Linear Regression via Maximum Likelihood	53
4.11	Overfitting	54
4.12	Linear Regression via MAP: Ridge Regression	54
4.13	Classification as Regression	55
4.14	Logistic Regression: Definition	56
4.15	Excursion: Gradient Descent	57
4.16	Logistic Regression via Maximum Likelihood	58
4.17	Summary	59
5	Lecture 5: Neural Networks – Perceptrons and Multi-Layer Perceptrons	61
5.1	Overview	61
5.2	Part A: Biological and Artificial Neural Networks	61
5.3	Definition of the Perceptron	62
5.4	Learning in Perceptrons	64
5.5	Perceptrons as Classifiers	65
5.6	Limitations of Perceptrons: The XOR Problem	66
5.7	Part B: Extension to Multi-Layer Perceptrons	67
5.8	Why Nonlinear Activation Functions Are Necessary	68
5.9	Learning Rules for Multi-Layer Perceptrons	69
5.10	Mechanistic View of Learning: Backpropagation	70
5.11	Perceptrons, MLPs and the XOR Problem	71
5.12	Prüfungsrelevante Kurzfassung	72
6	Lecture 6: Neural Networks – DNNs and Their Capabilities	73
6.1	Overview	73
6.2	Part C.1: Generalization of Backpropagation	74
6.3	Generalization of the Backpropagation Derivation	75
6.4	Properties of Multi-Layer Perceptrons	76

6.5	MLPs and the XOR Problem	77
6.6	Overfitting and Generalization Error for MLPs	78
6.7	Validation Sets for MLPs	79
6.8	Part C.2: MLPs and Maximum Likelihood Estimation	80
6.9	Part C.3: Generalization and Validation	81
6.10	Regularization and Early Stopping	82
6.11	Part C.4: Other Neural Networks	83
6.12	Prüfungsrelevante Kurzfassung	84
7	Lecture 7: Dimensionality Reduction with PCA	85
7.1	Overview	85
7.2	Preparation: Projection Operators	86
7.3	The One-Dimensional Problem: Finding a Line Through Data	87
7.4	Maximum Variance Approach in One Dimension	88
7.5	Optimal Solution in Terms of Eigenvectors	90
7.6	The General Problem: Finding a Low-Dimensional Hyperplane	91
7.7	Maximum Variance Approach in the General Case	92
7.8	Projection and Reconstruction	93
7.9	Explained Variance	94
7.10	Example Applications	95
7.11	Discussion of PCA	96
7.12	Prüfungsrelevante Kurzfassung	97
8	Lecture 8: Clustering – k-means Algorithms, Hierarchical Clustering and GMMs	98
8.1	Overview	98
8.2	The Problem of Finding Clusters in Data	99
8.3	k-means: Basic Idea	100
8.4	k-means: Definition of an Ad-hoc Objective	101
8.5	Derivation of Lloyd's Algorithm	102
8.6	Lloyd's Algorithm: Practical Procedure	103
8.7	Fuzzy-k-means	104
8.8	Hierarchical Clustering	105
8.9	Agglomerative Clustering	106
8.10	Gaussian Mixture Models	107
8.11	Latent Variables and Responsibilities in GMMs	108
8.12	Maximum Likelihood Parameter Estimation for GMMs	109
8.13	Free Energy and ELBO as Auxiliary Objectives	110
8.14	Update of the Mean in GMMs	111
8.15	Relation Between k-means and GMMs	112
8.16	Discussion	112
8.17	Prüfungsrelevante Kurzfassung	114

9	Lecture 9: Gaussian Mixture Models and the Expectation Maximization Algorithm	115
9.1	Overview	115
9.2	Gaussian Mixture Models Continued	116
9.3	Difference Between Log-Likelihood and Free Energy	117
9.4	Posterior Distributions as Optimal Choices for $q^{(n)}(c)$	119
9.5	Computation of Optimal Means Using an Iterative Algorithm	120
9.6	A First EM Algorithm for GMM Means	121
9.7	The Generality of the EM Algorithm	122
9.8	Derivations Do Not Depend on Specific Distribution Forms	123
9.9	Forms for Different Latent Variables	124
9.10	Definition of the EM Meta-Algorithm for Generative Models	125
9.11	Prüfungsrelevante Kurzfassung	126

1 Lecture 1: Introduction and Elements of Probabilistic Modelling

1.1 Introduction: What should an AI system do for us?

Notes

Die zentrale Frage der ersten Vorlesung ist:

Was soll ein AI-System eigentlich für uns tun?

Ein AI-System soll aus Daten sinnvolle Entscheidungen, Vorhersagen oder Schlussfolgerungen ableiten. Dabei soll es nicht nur feste Regeln ausführen, sondern auch mit Unsicherheit umgehen können.

Typische Aufgaben sind zum Beispiel:

- Objekte klassifizieren,
- zukünftige Werte vorhersagen,
- Muster in Daten erkennen,
- Entscheidungen unter Unsicherheit treffen.

Solution

In Machine Learning betrachtet man häufig Situationen, in denen Daten nicht eindeutig sind. Ein Bild, ein Messwert oder ein Text kann verrauscht, unvollständig oder mehrdeutig sein.

Deshalb ist es oft nicht sinnvoll, nur eine harte Entscheidung auszugeben, sondern eine probabilistische Antwort:

$$P(\text{Klasse} \mid \text{Daten}).$$

Das bedeutet: Das System gibt an, wie wahrscheinlich verschiedene mögliche Erklärungen oder Klassen sind, nachdem die Daten beobachtet wurden.

Answer

Ein AI-System soll aus Daten nützliche Entscheidungen oder Vorhersagen ableiten. Da reale Daten oft unsicher oder mehrdeutig sind, ist eine probabilistische Beschreibung sinnvoll:

AI-Systeme sollen unter Unsicherheit sinnvoll entscheiden.

1.2 Example: Classification

Notes

Bei der Klassifikation soll ein Objekt einer Klasse zugeordnet werden.

Wir bezeichnen:

x = Daten bzw. Beobachtung

und

y = Klasse bzw. Label.

Gesucht ist die Wahrscheinlichkeit:

$$P(y \mid x).$$

Calculation

Die Größe

$$P(y \mid x)$$

bedeutet:

Wie wahrscheinlich ist die Klasse y , wenn die Daten x beobachtet wurden?

Ein Klassifikator entscheidet sich meistens für die Klasse mit der größten Wahrscheinlichkeit:

$$\hat{y} = \arg \max_y P(y \mid x).$$

Dabei bedeutet $\hat{y}$ die vorhergesagte Klasse.

Solution

Beispielhaft kann man sich eine Klassifikation von Bildern vorstellen. Das System erhält ein Bild x und soll entscheiden, ob darauf zum Beispiel eine Katze, ein Hund oder ein Auto zu sehen ist.

Statt nur eine einzelne Antwort auszugeben, kann das System Wahrscheinlichkeiten für alle Klassen angeben:

$$P(\text{Katze} \mid x), \quad P(\text{Hund} \mid x), \quad P(\text{Auto} \mid x).$$

Wenn

$$P(\text{Katze} \mid x)$$

am größten ist, wird das Bild als Katze klassifiziert.

Answer

Klassifikation bedeutet:

Aus Daten x wird eine Klasse y vorhergesagt.

Probabilistisch formuliert sucht man:

$$P(y \mid x)$$

Die Entscheidung ist:

$$\hat{y} = \arg \max_y P(y \mid x)$$

1.3 Different Types of Learning

Notes

In Machine Learning unterscheidet man verschiedene Arten des Lernens. Wichtig sind vor allem:

- supervised learning,
- semi-supervised learning,
- unsupervised learning,
- reinforcement learning.

Solution

Beim *supervised learning* sind Trainingsdaten mit Labels gegeben. Das bedeutet, man hat Beispiele der Form

$$(x_i, y_i),$$

wobei x_i die Daten und y_i das zugehörige richtige Label ist.

Ziel ist es, aus diesen Beispielen ein Modell zu lernen, das auch für neue Daten gute Vorhersagen macht.

Solution

Beim *unsupervised learning* sind keine Labels gegeben. Man beobachtet nur Datenpunkte

$$x_1, x_2, \dots, x_n.$$

Das Ziel ist dann nicht, bekannte Labels vorherzusagen, sondern Strukturen in den Daten zu finden.

Typische Aufgaben sind:

- Clustering,
- Dimensionsreduktion,
- Dichteschätzung,
- Finden verborgener Muster.

Solution

Beim *semi-supervised learning* gibt es wenige gelabelte Daten und viele ungelabelte Daten. Man hat also zum Beispiel:

$$(x_1, y_1), \dots, (x_m, y_m)$$

mit Labels, aber zusätzlich viele Daten

$$x_{m+1}, \dots, x_n$$

ohne Labels.

Die Idee ist, beide Informationsquellen zu nutzen.

Solution

Beim *reinforcement learning* lernt ein Agent durch Interaktion mit einer Umgebung. Der Agent beobachtet einen Zustand, führt eine Aktion aus und erhält eine Belohnung. Ziel ist es, langfristig möglichst viel Belohnung zu sammeln.

Man kann sich das schematisch so vorstellen:

$$\text{Zustand} \longrightarrow \text{Aktion} \longrightarrow \text{Belohnung}.$$

Answer

Die wichtigsten Lernarten sind:

- **Supervised learning:** Lernen aus gelabelten Daten.
- **Semi-supervised learning:** Lernen aus wenigen gelabelten und vielen ungelabelten Daten.
- **Unsupervised learning:** Finden von Struktur in ungelabelten Daten.
- **Reinforcement learning:** Lernen durch Belohnung und Interaktion.

1.4 Probabilistic Answers as Optimal Inference

Notes

Ein wichtiger Gedanke der Vorlesung ist:

Probabilistische Antworten können als optimale Inferenz verstanden werden.

Inferenz bedeutet, aus Beobachtungen auf unbekannte Größen zu schließen.

Solution

Wenn ein AI-System Daten x beobachtet, möchte es auf eine unbekannte Größe y schließen. Das kann zum Beispiel eine Klasse, ein zukünftiger Wert oder eine verborgene Ursache

sein.

Statt nur einen einzelnen Wert auszugeben, beschreibt ein probabilistisches Modell die Unsicherheit über y durch eine Wahrscheinlichkeitsverteilung:

$$P(y \mid x).$$

Diese Verteilung fasst zusammen, welche Werte von y nach Beobachtung von x plausibel sind.

Answer

Probabilistische Inferenz bedeutet:

Aus beobachteten Daten x wird auf unbekannte Größen y geschlossen.

Das zentrale Objekt ist:

$$P(y \mid x)$$

1.5 Joint Probabilities and Conditional Probabilities

Notes

Die gemeinsame Wahrscheinlichkeit beschreibt, wie wahrscheinlich es ist, dass zwei Ereignisse gleichzeitig eintreten:

$$P(A, B).$$

Beispiel:

$$P(\text{krank, positiver Test}).$$

Das ist die Wahrscheinlichkeit, dass eine Person krank ist und gleichzeitig einen positiven Test hat.

Notes

Die bedingte Wahrscheinlichkeit beschreibt, wie wahrscheinlich ein Ereignis ist, wenn ein anderes Ereignis bereits bekannt ist:

$$P(A \mid B).$$

Beispiel:

$$P(\text{krank} \mid \text{positiver Test}).$$

Das bedeutet:

Wie wahrscheinlich ist es, krank zu sein, wenn der Test positiv ist?

Calculation

Die bedingte Wahrscheinlichkeit ist definiert durch:

$$P(A | B) = \frac{P(A,B)}{P(B)}$$

für $P(B) > 0$.

Umgestellt erhält man:

$$P(A,B) = P(A | B)P(B).$$

Diese Gleichung ist bereits die Produktregel.

Answer

Wichtige Begriffe:

$$P(A,B) = \text{gemeinsame Wahrscheinlichkeit von } A \text{ und } B$$

$$P(A | B) = \text{Wahrscheinlichkeit von } A \text{ gegeben } B$$

1.6 Sum Rule and Marginal Probabilities

Calculation

Die Summenregel erlaubt es, eine Variable aus einer gemeinsamen Wahrscheinlichkeit zu eliminieren.

Für diskrete Zufallsvariablen gilt:

$$P(A) = \sum_B P(A,B).$$

Das nennt man *Marginalisierung*.

Solution

Die Idee ist: Wenn B verschiedene mögliche Werte annehmen kann, summiert man über alle Möglichkeiten von B , um nur noch die Wahrscheinlichkeit von A zu erhalten.

Hat B zum Beispiel die möglichen Werte $b_1, b_2, \dots, b_n$, dann gilt:

$$P(A) = \sum_{i=1}^n P(A, b_i).$$

Answer

Die Summenregel lautet:

$$P(A) = \sum_B P(A,B)$$

Sie wird verwendet, um aus einer gemeinsamen Wahrscheinlichkeit eine Randwahrscheinlichkeit zu erhalten.

1.7 Product Rule

Calculation

Die Produktregel verbindet gemeinsame und bedingte Wahrscheinlichkeiten:

$$P(A,B) = P(A | B)P(B).$$

Genauso gilt:

$$P(A,B) = P(B | A)P(A).$$

Beide Ausdrücke beschreiben dieselbe gemeinsame Wahrscheinlichkeit.

Side Calculation

Man erhält die Produktregel direkt aus der Definition der bedingten Wahrscheinlichkeit:

$$P(A | B) = \frac{P(A,B)}{P(B)}.$$

Multiplikation mit $P(B)$ liefert:

$$P(A | B)P(B) = P(A,B).$$

Also:

$$P(A,B) = P(A | B)P(B).$$

Answer

Die Produktregel lautet:

$$P(A,B) = P(A | B)P(B)$$

und äquivalent:

$$P(A,B) = P(B | A)P(A)$$

1.8 Bayes' Rule

Calculation

Aus der Produktregel erhält man:

$$P(A,B) = P(A | B)P(B)$$

und auch:

$$P(A,B) = P(B | A)P(A).$$

Da beide Seiten dieselbe gemeinsame Wahrscheinlichkeit beschreiben, gilt:

$$P(A | B)P(B) = P(B | A)P(A).$$

Durch Umformen nach $P(A | B)$ erhält man Bayes' Regel:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Calculation

Für Klassifikationsprobleme schreibt man häufig:

$$P(y | x) = \frac{P(x | y)P(y)}{P(x)}.$$

Dabei ist:

$$P(y | x) \quad \text{Posterior,}$$

also die Wahrscheinlichkeit der Klasse y nach Beobachtung der Daten x .

$$P(x | y) \quad \text{Likelihood,}$$

also die Wahrscheinlichkeit, die Daten x zu beobachten, wenn die Klasse y gilt.

$$P(y) \quad \text{Prior,}$$

also das Vorwissen über die Klasse y .

$$P(x) \quad \text{Evidence,}$$

also der Normalisierungsfaktor.

Answer

Bayes' Regel lautet:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Für Klassifikation:

$$P(y | x) = \frac{P(x | y)P(y)}{P(x)}$$

Bayes' Regel erlaubt es, aus beobachteten Daten auf die wahrscheinlichste Ursache oder Klasse zu schließen.

1.9 Introductory Example: Pick-One-Berry-From-Two-Bowls

Solution

Ein typisches Einstiegsbeispiel ist folgendes:

Es gibt zwei Schüsseln mit unterschiedlichen Mischungen aus Beeren. Man zieht eine Beere und möchte daraus schließen, aus welcher Schüssel sie wahrscheinlich stammt.

Die relevante Wahrscheinlichkeit ist:

$$P(\text{Schüssel} \mid \text{Beere}).$$

Man beobachtet also die Beere und schließt rückwärts auf die wahrscheinlichere Ursache, nämlich die Schüssel.

Calculation

Mit Bayes' Regel erhält man:

$$P(\text{Schüssel} \mid \text{Beere}) = \frac{P(\text{Beere} \mid \text{Schüssel})P(\text{Schüssel})}{P(\text{Beere})}.$$

Die Interpretation ist:

- $P(\text{Schüssel})$ beschreibt das Vorwissen darüber, wie wahrscheinlich die jeweilige Schüssel ist.
- $P(\text{Beere} \mid \text{Schüssel})$ beschreibt die Wahrscheinlichkeit, diese Beere aus dieser Schüssel zu ziehen.
- $P(\text{Schüssel} \mid \text{Beere})$ ist die gesuchte Wahrscheinlichkeit nach Beobachtung der Beere.

Answer

Das Beeren-Schüssel-Beispiel ist ein Bayes-Beispiel:

Beobachtung: Beere $\implies$ gesucht: wahrscheinliche Schüssel

Mathematisch:

$P(\text{Schüssel} \mid \text{Beere}) = \frac{P(\text{Beere} \mid \text{Schüssel})P(\text{Schüssel})}{P(\text{Beere})}$

1.10 Example Algorithms: Outlook

Notes

Die erste Vorlesung gibt außerdem einen Ausblick auf Algorithmen, die später in der Lehrveranstaltung behandelt werden können.

Die Grundidee ist: Viele Machine-Learning-Algorithmen lassen sich als Verfahren verstehen, die Wahrscheinlichkeiten, Parameter oder Strukturen aus Daten lernen.

Solution

Beispiele für spätere Modellklassen und Algorithmen sind:

- Regressionsmodelle,
- logistische Regression,
- neuronale Netze,
- PCA zur Dimensionsreduktion,
- Clustering-Verfahren,
- Gaussian Mixture Models,
- Expectation-Maximization.

Die probabilistische Denkweise aus Lecture 1 bildet eine wichtige Grundlage für viele dieser späteren Methoden.

1.11 Evidence of Strong Models in Biological Vision

Solution

Die Vorlesung erwähnt auch die Adelson's Checkerboard Illusion. Diese optische Täuschung zeigt, dass Wahrnehmung nicht einfach nur rohe Bilddaten verarbeitet.

Das Gehirn interpretiert Sinneseindrücke unter Annahmen über Licht, Schatten und Objekte. Zwei Felder können physikalisch dieselbe Helligkeit haben, aber unterschiedlich wahrgenommen werden, weil das Gehirn Kontextinformationen einbezieht.

Answer

Die Botschaft für Machine Learning ist:

Auch biologische Intelligenz arbeitet mit starken Modellen der Welt und interpretiert Beobachtungen nicht rein passiv.

Das motiviert probabilistische Modelle: Ein gutes System nutzt Daten und Annahmen, um plausible Schlussfolgerungen zu ziehen.

1.12 Prüfungsrelevante Kurzfassung

Answer

Lecture 1 führt Machine Learning als Entscheidungsfindung unter Unsicherheit ein. Besonders wichtig ist Klassifikation:

$$x \mapsto y.$$

Probabilistisch formuliert sucht man:

$$P(y \mid x).$$

Dafür braucht man:

- gemeinsame Wahrscheinlichkeiten $P(A, B)$,
- bedingte Wahrscheinlichkeiten $P(A \mid B)$,
- die Summenregel,
- die Produktregel,
- Bayes' Regel.

Die wichtigste Formel ist:

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}$$

Bayes' Regel ist zentral, weil sie erlaubt, aus beobachteten Daten auf wahrscheinliche Ursachen oder Klassen zu schließen.

2 Lecture 2: Probabilistic Calculus

2.1 Overview

Notes

Lecture 2 erweitert die probabilistischen Grundlagen aus Lecture 1.

Die zentralen Themen sind:

- allgemeine Form von Produktregel, Summenregel und Bayes' Regel,
- kontinuierliche Zufallsvariablen und Wahrscheinlichkeitsdichten,
- Produktregel, Summenregel und Bayes' Regel für kontinuierliche Variablen,
- Dichteschätzung mit Histogrammen,
- Erwartungswerte,
- Approximation von Erwartungswerten durch Sampling,

- Mittelwert, Varianz, Kovarianz und Korrelation,
- wichtige Wahrscheinlichkeitsverteilungen.

Answer

Die Kernaussage von Lecture 2 ist:

Wahrscheinlichkeitsrechnung liefert die Rechenregeln, mit denen Unsicherheit mathematisch verarbeitet wird.

Diese Regeln bilden die Grundlage für probabilistische Modelle im Machine Learning.

2.2 Generality of the Product Rule, Sum Rule and Bayes' Rule

Notes

Die drei wichtigsten Regeln aus Lecture 1 gelten nicht nur für das Beeren-Beispiel, sondern allgemein für Zufallsvariablen und Ereignisse.

Die drei Grundregeln sind:

- Produktregel,
- Summenregel,
- Bayes' Regel.

Calculation

Für zwei Zufallsvariablen X und Y beschreibt die gemeinsame Wahrscheinlichkeit

$$P(X,Y)$$

die Wahrscheinlichkeit, dass X und Y gemeinsam bestimmte Werte annehmen.

Die bedingte Wahrscheinlichkeit ist definiert durch:

$$P(X | Y) = \frac{P(X,Y)}{P(Y)}$$

für $P(Y) > 0$.

Daraus folgt durch Umstellen die Produktregel:

$$P(X,Y) = P(X | Y)P(Y).$$

Genauso gilt:

$$P(X,Y) = P(Y | X)P(X).$$

Calculation

Da beide Ausdrücke dieselbe gemeinsame Wahrscheinlichkeit beschreiben, gilt:

$$P(X | Y)P(Y) = P(Y | X)P(X).$$

Auflösen nach $P(X | Y)$ ergibt Bayes' Regel:

$$P(X | Y) = \frac{P(Y | X)P(X)}{P(Y)}.$$

Calculation

Die Summenregel erlaubt es, eine Variable aus einer gemeinsamen Wahrscheinlichkeit zu entfernen.

Für diskrete Zufallsvariablen gilt:

$$P(X) = \sum_Y P(X,Y).$$

Das nennt man Marginalisierung.

Answer

Die wichtigsten diskreten Regeln sind:

$$P(X,Y) = P(X | Y)P(Y)$$

$$P(X) = \sum_Y P(X,Y)$$

$$P(X | Y) = \frac{P(Y | X)P(X)}{P(Y)}$$

2.3 Continuous Random Variables and Probability Density Functions

Notes

Viele Größen in Machine Learning sind nicht diskret, sondern kontinuierlich.

Beispiele:

- Körpergröße,
- Temperatur,
- Gewicht,
- Zeit,
- Messwerte,

- Positionen in einem Bild,
- Parameter eines Modells.

Eine kontinuierliche Zufallsvariable kann unendlich viele Werte in einem Intervall annehmen.

Solution

Bei diskreten Zufallsvariablen kann man einzelnen Werten direkte Wahrscheinlichkeiten zuordnen:

$$P(X = x).$$

Bei kontinuierlichen Zufallsvariablen ist die Wahrscheinlichkeit für einen exakt einzelnen Wert normalerweise:

$$P(X = x) = 0.$$

Stattdessen betrachtet man Wahrscheinlichkeiten für Intervalle:

$$P(a \leq X \leq b).$$

Diese werden über eine Wahrscheinlichkeitsdichte $p(x)$ berechnet:

$$P(a \leq X \leq b) = \int_a^b p(x) dx.$$

Calculation

Eine Wahrscheinlichkeitsdichte $p(x)$ muss zwei Eigenschaften erfüllen:

$$p(x) \geq 0$$

für alle x , und

$$\int_{-\infty}^{\infty} p(x) dx = 1.$$

Die Gesamtfläche unter der Dichtefunktion ist also 1.

Answer

Für kontinuierliche Zufallsvariablen gilt:

$$P(a \leq X \leq b) = \int_a^b p(x) dx$$

Die Funktion $p(x)$ ist keine Wahrscheinlichkeit an einem Punkt, sondern eine Dichte.
Wichtig:

$$P(X = x) = 0 \quad \text{für kontinuierliche } X.$$

2.4 Product Rule, Sum Rule and Bayes' Rule for Continuous Random Variables

Notes

Die bekannten Rechenregeln gelten auch für kontinuierliche Zufallsvariablen.
Der Unterschied ist:

- Summen werden durch Integrale ersetzt.
- Wahrscheinlichkeiten werden durch Dichten ersetzt.

Calculation

Für kontinuierliche Zufallsvariablen X und Y verwendet man die gemeinsame Dichte

$$p(x,y).$$

Die bedingte Dichte ist:

$$p(x | y) = \frac{p(x,y)}{p(y)}.$$

Daraus folgt die Produktregel:

$$p(x,y) = p(x | y)p(y).$$

Ebenso gilt:

$$p(x,y) = p(y | x)p(x).$$

Calculation

Die kontinuierliche Summenregel lautet:

$$p(x) = \int p(x,y) dy.$$

Mit der Produktregel:

$$p(x) = \int p(x | y)p(y) dy.$$

Das Integral marginalisiert die Variable Y heraus.

Calculation

Bayes' Regel für kontinuierliche Zufallsvariablen lautet:

$$p(x | y) = \frac{p(y | x)p(x)}{p(y)}.$$

Der Nenner kann wieder durch Marginalisierung berechnet werden:

$$p(y) = \int p(y | x)p(x) dx.$$

Damit:

$$p(x | y) = \frac{p(y | x)p(x)}{\int p(y | x)p(x) dx}.$$

Answer

Für kontinuierliche Zufallsvariablen gelten die analogen Regeln:

$$p(x,y) = p(x | y)p(y)$$

$$p(x) = \int p(x,y) dy$$

$$p(x | y) = \frac{p(y | x)p(x)}{p(y)}$$

Der wichtigste Unterschied zur diskreten Version ist:

$$\sum \rightarrow \int$$

2.5 Density Estimation Using Histograms

Notes

In vielen Anwendungen kennt man die wahre Dichte $p(x)$ nicht.

Man hat nur Daten:

$$x^{(1)}, x^{(2)}, \dots, x^{(N)}.$$

Aus diesen Daten möchte man eine Schätzung für die zugrundeliegende Dichte konstruieren. Das nennt man *density estimation*.

Solution

Eine einfache Methode zur Dichteschätzung ist ein Histogramm.

Man teilt den Wertebereich in Intervalle, sogenannte Bins, ein.

Für jeden Bin zählt man, wie viele Datenpunkte in diesem Intervall liegen.

Wenn ein Bin die Breite Δ hat und n_i Datenpunkte enthält, dann kann man die Dichte in diesem Bin approximieren durch:

$$\hat{p}_i = \frac{n_i}{N\Delta}.$$

Dabei ist:

- n_i die Anzahl der Datenpunkte im Bin,
- N die Gesamtzahl der Datenpunkte,

- Δ die Breite des Bins.

Calculation

Die Normierung funktioniert, weil:

$$\sum_i \hat{p}_i \Delta = \sum_i \frac{n_i}{N\Delta} \Delta.$$

Kürzen von Δ ergibt:

$$\sum_i \hat{p}_i \Delta = \frac{1}{N} \sum_i n_i.$$

Da alle Datenpunkte auf die Bins verteilt werden, gilt:

$$\sum_i n_i = N.$$

Also:

$$\sum_i \hat{p}_i \Delta = 1.$$

Answer

Histogramme schätzen eine Dichte durch Zählen von Datenpunkten in Intervallen:

$$\hat{p}_i = \frac{n_i}{N\Delta}$$

Kleine Bin-Breiten liefern detaillierte, aber oft verrauschte Schätzungen.

Große Bin-Breiten liefern glattere, aber gröbere Schätzungen.

2.6 Expectation Values

Notes

Der Erwartungswert beschreibt den durchschnittlichen Wert einer Zufallsvariable unter ihrer Wahrscheinlichkeitsverteilung.

Man schreibt:

$$\mathbb{E}[X].$$

Calculation

Für eine diskrete Zufallsvariable X mit möglichen Werten x_i gilt:

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i).$$

Allgemeiner für eine Funktion $f(X)$:

$$\mathbb{E}[f(X)] = \sum_i f(x_i)P(X = x_i).$$

Calculation

Für eine kontinuierliche Zufallsvariable mit Dichte $p(x)$ gilt:

$$\mathbb{E}[X] = \int xp(x) dx.$$

Allgemeiner:

$$\mathbb{E}[f(X)] = \int f(x)p(x) dx.$$

Answer

Der Erwartungswert ist ein gewichteter Mittelwert.

Diskret:

$$\mathbb{E}[f(X)] = \sum_i f(x_i)P(X = x_i)$$

Kontinuierlich:

$$\mathbb{E}[f(X)] = \int f(x)p(x) dx$$

2.7 Sampling Approximation of Expectation Values

Notes

Erwartungswerte können oft nicht exakt berechnet werden.

Wenn man aber Samples

$$x^{(1)}, x^{(2)}, \dots, x^{(N)}$$

aus der Verteilung $p(x)$ hat, kann man Erwartungswerte durch Mittelwerte über die Samples approximieren.

Calculation

Der exakte Erwartungswert einer Funktion $f(X)$ ist:

$$\mathbb{E}[f(X)] = \int f(x)p(x) dx.$$

Mit Samples $x^{(1)}, \dots, x^{(N)}$ approximiert man:

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{n=1}^N f(x^{(n)}).$$

Diese Formel ist die Sampling-Approximation des Erwartungswertes.

Solution

Die Idee ist:

Werte, die unter $p(x)$ häufig auftreten, kommen auch häufiger in den Samples vor.

Dadurch ersetzt die Summe über Samples näherungsweise das Integral über die Dichte.

Für große N wird die Approximation typischerweise besser.

Answer

Die Sampling-Approximation lautet:

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{n=1}^N f(x^{(n)})$$

Sie ist zentral für Monte-Carlo-Methoden und viele numerische Verfahren im Machine Learning.

2.8 Mean, Variance, Covariance and Correlation

Notes

Erwartungswerte werden verwendet, um wichtige Kenngrößen von Verteilungen zu definieren:

- mean,
- variance,
- covariance,
- correlation.

Calculation

Der Mittelwert oder Erwartungswert einer Zufallsvariable X ist:

$$\mu = \mathbb{E}[X].$$

Für kontinuierliche Variablen:

$$\mu = \int x p(x) dx.$$

Calculation

Die Varianz misst die mittlere quadratische Abweichung vom Mittelwert:

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2].$$

Äquivalent gilt:

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

Die Standardabweichung ist:

$$\sigma = \sqrt{\text{Var}(X)}.$$

Calculation

Die Kovarianz zweier Zufallsvariablen X und Y ist:

$$\text{Cov}(X,Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)].$$

Äquivalent:

$$\text{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Sie misst, ob zwei Variablen gemeinsam größer oder kleiner werden.

Calculation

Die Korrelation ist die normierte Kovarianz:

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}.$$

Dabei gilt:

$$-1 \leq \rho_{X,Y} \leq 1.$$

Interpretation:

- $\rho_{X,Y} > 0$: positive lineare Beziehung,
- $\rho_{X,Y} < 0$: negative lineare Beziehung,
- $\rho_{X,Y} = 0$: keine lineare Korrelation.

Answer

Die wichtigsten Größen sind:

$$\mu = \mathbb{E}[X]$$

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

$$\text{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

2.9 Important Probability Distributions

Notes

In Machine Learning treten bestimmte Wahrscheinlichkeitsverteilungen immer wieder auf. Sie dienen als Modelle für Daten, Unsicherheit, Fehler oder Parameter.

Solution

Die **Bernoulli-Verteilung** modelliert ein binäres Ereignis:

$$X \in \{0,1\}.$$

Mit Parameter p gilt:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

Kompakter:

$$P(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0,1\}.$$

Solution

Die **kategoriale Verteilung** modelliert eine Zufallsvariable mit mehreren möglichen Klassen:

$$X \in \{1, \dots, K\}.$$

Für Klassenwahrscheinlichkeiten $\pi_1, \dots, \pi_K$ gilt:

$$P(X = k) = \pi_k$$

mit

$$\sum_{k=1}^K \pi_k = 1.$$

Solution

Die **Binomialverteilung** modelliert die Anzahl der Erfolge in N unabhängigen Bernoulli-Experimenten.

Für k Erfolge gilt:

$$P(X = k) = \binom{N}{k} p^k (1 - p)^{N-k}.$$

Solution

Die **Normalverteilung** oder Gauß-Verteilung ist eine der wichtigsten kontinuierlichen Verteilungen.

Für Mittelwert μ und Varianz σ^2 gilt:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Sie ist wichtig für Messfehler, natürliche Streuung und viele approximative Methoden.

Solution

Die **multivariate Normalverteilung** verallgemeinert die Normalverteilung auf Vektoren.

Für $x \in \mathbb{R}^D$, Mittelwertvektor μ und Kovarianzmatrix Σ gilt:

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1} (x - \mu)\right).$$

Sie modelliert kontinuierliche, mehrdimensionale Daten.

Answer

Wichtige Wahrscheinlichkeitsverteilungen sind:

- **Bernoulli-Verteilung:** binäre Ereignisse,
- **kategoriale Verteilung:** mehrere Klassen,
- **Binomialverteilung:** Anzahl von Erfolgen,
- **Normalverteilung:** kontinuierliche Streuung um einen Mittelwert,
- **multivariate Normalverteilung:** kontinuierliche Vektordaten.

Diese Verteilungen sind Grundbausteine probabilistischer Modelle.

2.10 Prüfungsrelevante Kurzfassung

Answer

Lecture 2 behandelt den probabilistischen Kalkül, also die mathematischen Rechenregeln für Wahrscheinlichkeiten.

Die wichtigsten Punkte sind:

- Produktregel:

$$P(X, Y) = P(X \mid Y)P(Y).$$

- Summenregel:

$$P(X) = \sum_Y P(X, Y).$$

- Bayes' Regel:

$$P(X | Y) = \frac{P(Y | X)P(X)}{P(Y)}.$$

- Für kontinuierliche Variablen ersetzt man Summen durch Integrale:

$$p(x) = \int p(x,y) dy.$$

- Wahrscheinlichkeiten für kontinuierliche Variablen werden über Dichten berechnet:

$$P(a \leq X \leq b) = \int_a^b p(x) dx.$$

- Erwartungswert:

$$\mathbb{E}[f(X)] = \int f(x)p(x) dx.$$

- Sampling-Approximation:

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{n=1}^N f(x^{(n)}).$$

- Varianz:

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

- Kovarianz:

$$\text{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

- Korrelation:

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}.$$

Die zentrale Botschaft lautet:

Probabilistic calculus ist das Rechenwerkzeug für Machine Learning unter Unsicherheit.

3 Lecture 3: Probabilities and Estimators

3.1 Overview

Notes

Lecture 3 behandelt den Übergang von Wahrscheinlichkeitsrechnung zu statistischer Schätzung.

Die zentrale Frage lautet:

Wie kann man unbekannte Parameter einer Verteilung aus Daten schätzen?

Wichtige Themen sind:

- Parameterschätzung für Gleichverteilungen,
- Methode der Momente,
- Funktionen von Zufallsvariablen,
- i.i.d. Zufallsvariablen,
- statistische Schätzer,
- Bias und Varianz eines Schätzers,
- Bias und Varianz des Stichprobenmittelwerts,
- unverzerzte Stichprobenvarianz,
- Maximum-Likelihood-Schätzer,
- Maximum-a-posteriori-Schätzer,
- Bayes-Schätzer,
- Standardableitungen für ML- und MAP-Schätzer.

Answer

Die Kernaussage von Lecture 3 ist:

Ein Schätzer ist eine Funktion der Daten, die einen unbekannten Parameter approximiert.

In Machine Learning bedeutet Lernen häufig genau das: Aus Trainingsdaten werden Modellparameter geschätzt.

3.2 Parameter Estimation for Uniform Distributions

Notes

Als einfaches Beispiel betrachten wir eine Gleichverteilung.

Sei

$$X \sim \text{Unif}(a,b).$$

Dann ist X gleichverteilt auf dem Intervall $[a,b]$.

Die Dichte lautet:

$$p(x \mid a,b) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b, \\ 0, & \text{sonst.} \end{cases}$$

Die unbekannten Parameter sind:

$$a \quad \text{und} \quad b.$$

Solution

Angenommen, wir beobachten Daten

$$x^{(1)}, x^{(2)}, \dots, x^{(N)}$$

und nehmen an, dass diese Daten aus einer Gleichverteilung stammen.

Dann ist die Aufgabe:

Schätze die Intervallgrenzen a und b .

Dafür gibt es verschiedene Ansätze. Zwei einfache Methoden sind:

- heuristische Schätzung mit Minimum und Maximum der Daten,
- Schätzung mit der Methode der Momente.

3.3 Heuristic Estimation Using Minimum and Maximum

Notes

Eine naheliegende Idee ist:

$$\hat{a} = \min_{1 \leq n \leq N} x^{(n)}$$

und

$$\hat{b} = \max_{1 \leq n \leq N} x^{(n)}.$$

Man verwendet also den kleinsten beobachteten Datenpunkt als Schätzer für die untere Grenze und den größten beobachteten Datenpunkt als Schätzer für die obere Grenze.

Calculation

Für Daten

$$x^{(1)}, \dots, x^{(N)}$$

definieren wir:

$$x_{\min} = \min_{1 \leq n \leq N} x^{(n)}$$

und

$$x_{\max} = \max_{1 \leq n \leq N} x^{(n)}.$$

Der heuristische Schätzer lautet dann:

$$\hat{a}_{\min\max} = x_{\min}$$

und

$$\hat{b}_{\min\max} = x_{\max}.$$

Solution

Diese Methode ist intuitiv, weil bei einer Gleichverteilung auf $[a,b]$ alle Datenpunkte innerhalb dieses Intervalls liegen müssen.

Der kleinste beobachtete Wert ist daher ein Hinweis auf die untere Grenze, und der größte beobachtete Wert ist ein Hinweis auf die obere Grenze.

Allerdings gilt typischerweise:

$$x_{\min} > a$$

und

$$x_{\max} < b.$$

Das bedeutet: Für endliche Stichproben unterschätzt diese Methode oft die Intervalllänge.

Answer

Der Min-Max-Schätzer für eine Gleichverteilung ist:

$$\hat{a} = x_{\min}$$

$$\hat{b} = x_{\max}$$

Er ist einfach und plausibel, verwendet aber nur die beiden extremen Datenpunkte.

3.4 Estimation Using the Method of Moments

Notes

Die Methode der Momente schätzt Parameter, indem theoretische Momente einer Verteilung mit empirischen Momenten der Daten gleichgesetzt werden.

Für eine Gleichverteilung $X \sim \text{Unif}(a,b)$ gilt:

$$\mathbb{E}[X] = \frac{a+b}{2}$$

und

$$\text{Var}(X) = \frac{(b-a)^2}{12}.$$

Calculation

Zuerst berechnet man aus den Daten den empirischen Mittelwert:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x^{(n)}.$$

Dann berechnet man die empirische Varianz mit Faktor $1/N$:

$$v = \frac{1}{N} \sum_{n=1}^N \left(x^{(n)} - \bar{x} \right)^2.$$

Die Methode der Momente setzt nun:

$$\bar{x} = \frac{a+b}{2}$$

und

$$v = \frac{(b-a)^2}{12}.$$

Calculation

Aus

$$\bar{x} = \frac{a+b}{2}$$

folgt:

$$a+b = 2\bar{x}.$$

Aus

$$v = \frac{(b-a)^2}{12}$$

folgt:

$$(b-a)^2 = 12v.$$

Also:

$$b-a = \sqrt{12v}.$$

Setze

$$m = \bar{x}$$

und

$$d = \frac{b-a}{2}.$$

Dann gilt:

$$d = \frac{\sqrt{12v}}{2} = \sqrt{3v}.$$

Damit erhält man:

$$\hat{a}_{\text{MoM}} = \bar{x} - \sqrt{3v}$$

und

$$\hat{b}_{\text{MoM}} = \bar{x} + \sqrt{3v}.$$

Answer

Für $X \sim \text{Unif}(a,b)$ liefert die Methode der Momente:

$$\hat{a}_{\text{MoM}} = \bar{x} - \sqrt{3v}$$

$$\hat{b}_{\text{MoM}} = \bar{x} + \sqrt{3v}$$

wobei

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x^{(n)}$$

und

$$v = \frac{1}{N} \sum_{n=1}^N \left(x^{(n)} - \bar{x} \right)^2.$$

3.5 Which Estimation Method Is Better?

Notes

Für die Gleichverteilung haben wir zwei Schätzmethoden betrachtet:

- Min-Max-Schätzung,
- Methode der Momente.

Die Frage ist:

Welche Methode ist besser?

Solution

Die Antwort hängt davon ab, welches Kriterium man betrachtet.

Die Min-Max-Schätzung verwendet nur zwei Datenpunkte:

$$x_{\min} \quad \text{und} \quad x_{\max}.$$

Dadurch ist sie sehr direkt für die Intervallgrenzen, aber auch stark von Extremwerten abhängig.

Die Methode der Momente verwendet dagegen alle Datenpunkte über Mittelwert und Varianz:

$$\bar{x} \quad \text{und} \quad v.$$

Dadurch nutzt sie mehr Information aus den Daten, kann aber Intervallgrenzen liefern, die nicht exakt zu Minimum und Maximum der beobachteten Daten passen.

Answer

Es gibt nicht immer eine universell beste Methode.

- Der Min-Max-Schätzer ist natürlich für Intervallgrenzen und eng mit der Maximum-Likelihood-Idee für die Gleichverteilung verbunden.
- Die Methode der Momente ist systematisch und verwendet Mittelwert und Varianz, also Informationen aus allen Datenpunkten.
- Für endliche Stichproben kann man Methoden anhand von Bias, Varianz und mittlerem quadratischem Fehler vergleichen.

Wichtige Kriterien sind daher:

Bias, Varianz und Mean Squared Error.

3.6 Mean and Variance of Functions of Random Numbers

Notes

Oft interessiert man sich nicht nur für eine Zufallsvariable X , sondern für eine Funktion dieser Zufallsvariable:

$$Y = f(X).$$

Dann sind auch Mittelwert und Varianz von Y relevant.

Calculation

Für eine diskrete Zufallsvariable gilt:

$$\mathbb{E}[f(X)] = \sum_x f(x)P(X = x).$$

Für eine kontinuierliche Zufallsvariable mit Dichte $p(x)$ gilt:

$$\mathbb{E}[f(X)] = \int f(x)p(x) dx.$$

Die Varianz von $f(X)$ ist:

$$\text{Var}(f(X)) = \mathbb{E} \left[(f(X) - \mathbb{E}[f(X)])^2 \right].$$

Calculation

Äquivalent kann man die Varianz schreiben als:

$$\text{Var}(f(X)) = \mathbb{E}[f(X)^2] - \mathbb{E}[f(X)]^2.$$

Diese Darstellung ist oft rechnerisch einfacher.

Answer

Für eine Funktion $Y = f(X)$ gilt:

$$\boxed{\mathbb{E}[Y] = \mathbb{E}[f(X)]}$$

und

$$\boxed{\text{Var}(Y) = \mathbb{E}[f(X)^2] - \mathbb{E}[f(X)]^2.}$$

3.7 i.i.d. Random Variables

Notes

In der Statistik und im Machine Learning nimmt man häufig an, dass Daten *i.i.d.* sind. Die Abkürzung steht für:

independent and identically distributed.

Das bedeutet:

- Die Zufallsvariablen sind unabhängig.
- Alle Zufallsvariablen haben dieselbe Verteilung.

Solution

Für Daten

$$X_1, \dots, X_N$$

bedeutet i.i.d.:

$$X_1, \dots, X_N \sim p(x)$$

und die gemeinsame Verteilung faktorisiert:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n).$$

Diese Faktorisierung ist extrem wichtig für Maximum-Likelihood-Schätzung.

Answer

Die i.i.d.-Annahme lautet:

$$X_1, \dots, X_N \text{ sind unabhängig und identisch verteilt.}$$

Dann gilt:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n).$$

3.8 Statistical Estimators

Notes

Ein statistischer Schätzer ist eine Funktion der Daten, die einen unbekannten Parameter einer Verteilung approximieren soll.

Sei θ ein unbekannter Parameter und seien

$$X_1, \dots, X_N$$

Zufallsvariablen, aus denen die Daten entstehen.

Ein Schätzer ist dann eine Funktion

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_N).$$

Solution

Wichtig ist:

Da der Schätzer von Zufallsvariablen abhängt, ist der Schätzer selbst wieder eine Zufallsvariable.

Für konkrete beobachtete Daten

$$x_1, \dots, x_N$$

erhält man einen konkreten Schätzwert:

$$\hat{\theta}(x_1, \dots, x_N).$$

Vor der Beobachtung der Daten ist $\hat{\theta}$ aber zufällig.

Answer

Ein Schätzer ist:

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_N)$$

Er ist eine Funktion der Daten und damit selbst eine Zufallsvariable.

Ein guter Schätzer sollte den unbekannten Parameter θ möglichst gut approximieren.

3.9 Bias and Variance of Estimators

Notes

Zwei zentrale Eigenschaften eines Schätzers sind:

- Bias,
- Varianz.

Der Bias misst, ob ein Schätzer systematisch danebenliegt.

Die Varianz misst, wie stark der Schätzer zwischen verschiedenen Stichproben schwankt.

Calculation

Der Bias eines Schätzers $\hat{\theta}$ für einen Parameter θ ist:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

Ist

$$\mathbb{E}[\hat{\theta}] = \theta,$$

dann ist der Schätzer unverzerrt oder unbiased.

Also:

$$\text{Bias}(\hat{\theta}) = 0 \iff \hat{\theta} \text{ ist unverzerrt.}$$

Calculation

Die Varianz eines Schätzers ist:

$$\text{Var}(\hat{\theta}) = \mathbb{E} \left[\left(\hat{\theta} - \mathbb{E}[\hat{\theta}] \right)^2 \right].$$

Sie beschreibt, wie stark die Schätzung über verschiedene mögliche Datensätze schwankt.

Calculation

Ein häufiges Fehlermaß ist der mittlere quadratische Fehler:

$$\text{MSE}(\hat{\theta}) = \mathbb{E} \left[\left(\hat{\theta} - \theta \right)^2 \right].$$

Dieser zerfällt in Bias und Varianz:

$$\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

Answer

Für einen Schätzer $\hat{\theta}$ gilt:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$$

$$\text{Var}(\hat{\theta}) = \mathbb{E} \left[\left(\hat{\theta} - \mathbb{E}[\hat{\theta}] \right)^2 \right]$$

$$\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2$$

3.10 Bias and Variance of the Sample Mean Estimator

Notes

Sei

$$X_1, \dots, X_N$$

eine i.i.d. Stichprobe mit

$$\mathbb{E}[X_i] = \mu$$

und

$$\text{Var}(X_i) = \sigma^2.$$

Der Stichprobenmittelwert ist:

$$\bar{X} = \frac{1}{N} \sum_{n=1}^N X_n.$$

Calculation

Erwartungswert des Stichprobenmittelwerts:

$$\mathbb{E}[\bar{X}] = \mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N X_n \right].$$

Wegen Linearität des Erwartungswerts:

$$\mathbb{E}[\bar{X}] = \frac{1}{N} \sum_{n=1}^N \mathbb{E}[X_n].$$

Da alle X_n denselben Erwartungswert μ haben:

$$\mathbb{E}[\bar{X}] = \frac{1}{N} \sum_{n=1}^N \mu = \frac{1}{N} N \mu = \mu.$$

Also:

$$\text{Bias}(\bar{X}) = \mathbb{E}[\bar{X}] - \mu = 0.$$

Calculation

Varianz des Stichprobenmittelwerts:

$$\text{Var}(\bar{X}) = \text{Var} \left(\frac{1}{N} \sum_{n=1}^N X_n \right).$$

Konstanten können quadratisch herausgezogen werden:

$$\text{Var}(\bar{X}) = \frac{1}{N^2} \text{Var} \left(\sum_{n=1}^N X_n \right).$$

Wegen Unabhängigkeit gilt:

$$\text{Var} \left(\sum_{n=1}^N X_n \right) = \sum_{n=1}^N \text{Var}(X_n).$$

Da alle X_n Varianz σ^2 haben:

$$\text{Var}(\bar{X}) = \frac{1}{N^2} \sum_{n=1}^N \sigma^2 = \frac{1}{N^2} N \sigma^2 = \frac{\sigma^2}{N}.$$

Answer

Für den Stichprobenmittelwert gilt:

$$\mathbb{E}[\bar{X}] = \mu$$

$$\text{Bias}(\bar{X}) = 0$$

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{N}$$

Der Stichprobenmittelwert ist also unverzerrt, und seine Varianz sinkt mit wachsender Stichprobengröße N .

3.11 Bias of the Naive Sample Variance Estimator

Notes

Eine naive Schätzung der Varianz ist:

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \bar{X})^2.$$

Diese Formel sieht natürlich aus, ist aber für die wahre Varianz σ^2 verzerrt.

Calculation

Man kann zeigen:

$$\mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N (X_n - \bar{X})^2 \right] = \frac{N-1}{N} \sigma^2.$$

Daher ist der Bias:

$$\text{Bias}(\hat{\sigma}_N^2) = \mathbb{E}[\hat{\sigma}_N^2] - \sigma^2$$

also:

$$\text{Bias}(\hat{\sigma}_N^2) = \frac{N-1}{N} \sigma^2 - \sigma^2.$$

Zusammenfassen ergibt:

$$\text{Bias}(\hat{\sigma}_N^2) = -\frac{1}{N} \sigma^2.$$

Answer

Der naive Varianzschätzer

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \bar{X})^2$$

ist verzerrt:

$$\mathbb{E}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma^2$$

und damit:

$$\text{Bias}(\hat{\sigma}_N^2) = -\frac{\sigma^2}{N}.$$

3.12 An Unbiased Estimator for the Sample Variance

Notes

Um einen unverzerrten Varianzschätzer zu erhalten, ersetzt man den Faktor $1/N$ durch $1/(N-1)$.

Der korrigierte Schätzer lautet:

$$S^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \bar{X})^2.$$

Calculation

Aus der vorherigen Beziehung wissen wir:

$$\mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N (X_n - \bar{X})^2 \right] = \frac{N-1}{N} \sigma^2.$$

Multiplizieren wir den naiven Schätzer mit

$$\frac{N}{N-1},$$

erhalten wir:

$$\frac{N}{N-1} \cdot \frac{1}{N} \sum_{n=1}^N (X_n - \bar{X})^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \bar{X})^2.$$

Daher gilt:

$$\mathbb{E}[S^2] = \sigma^2.$$

Answer

Der unverzernte Schätzer für die Varianz ist:

$$S^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \bar{X})^2$$

Für ihn gilt:

$$\mathbb{E}[S^2] = \sigma^2.$$

Die Korrektur von N zu $N-1$ heißt Bessel-Korrektur.

3.13 Maximum Likelihood Estimators

Notes

Maximum Likelihood Estimation, kurz MLE, ist eine der wichtigsten Methoden zur Parameterschätzung.

Idee:

Wähle den Parameter, unter dem die beobachteten Daten am wahrscheinlichsten sind.

Sei θ ein Parameter und seien die Daten

$$x_1, \dots, x_N.$$

Calculation

Die Likelihood ist die Wahrscheinlichkeit der beobachteten Daten als Funktion des Parameters:

$$L(\theta) = p(x_1, \dots, x_N \mid \theta).$$

Unter der i.i.d.-Annahme gilt:

$$L(\theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

Der Maximum-Likelihood-Schätzer ist:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} L(\theta).$$

Calculation

Da Produkte oft schwer zu optimieren sind, verwendet man meistens die Log-Likelihood:

$$\ell(\theta) = \log L(\theta).$$

Wegen der Monotonie des Logarithmus gilt:

$$\arg \max_{\theta} L(\theta) = \arg \max_{\theta} \ell(\theta).$$

Unter der i.i.d.-Annahme:

$$\ell(\theta) = \log \prod_{n=1}^N p(x_n | \theta) = \sum_{n=1}^N \log p(x_n | \theta).$$

Answer

Der Maximum-Likelihood-Schätzer ist:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(x_1, \dots, x_N | \theta)$$

Für i.i.d. Daten:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{n=1}^N p(x_n | \theta)$$

Äquivalent maximiert man die Log-Likelihood:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n | \theta).$$

3.14 Maximum A-Posteriori Estimators

Notes

Maximum a-posteriori Estimation, kurz MAP, erweitert Maximum Likelihood um Vorwissen über den Parameter.

Dieses Vorwissen wird als Prior modelliert:

$$p(\theta).$$

Gesucht ist nun der wahrscheinlichste Parameter nach Beobachtung der Daten:

$$p(\theta | x_1, \dots, x_N).$$

Calculation

Nach Bayes' Regel gilt:

$$p(\theta \mid x_1, \dots, x_N) = \frac{p(x_1, \dots, x_N \mid \theta)p(\theta)}{p(x_1, \dots, x_N)}.$$

Der Nenner hängt nicht von θ ab.

Daher ist:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid x_1, \dots, x_N)$$

äquivalent zu:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(x_1, \dots, x_N \mid \theta)p(\theta).$$

Calculation

Für i.i.d. Daten gilt:

$$p(x_1, \dots, x_N \mid \theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

Also:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\prod_{n=1}^N p(x_n \mid \theta) \right] p(\theta).$$

In Log-Form:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{n=1}^N \log p(x_n \mid \theta) + \log p(\theta) \right].$$

Answer

Der MAP-Schätzer ist:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid x_1, \dots, x_N)$$

Äquivalent:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(x_1, \dots, x_N \mid \theta)p(\theta)$$

In Log-Form:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{n=1}^N \log p(x_n \mid \theta) + \log p(\theta) \right].$$

3.15 Bayesian Estimators

Notes

Bayessche Schätzung geht noch einen Schritt weiter als MAP.

Bei ML und MAP wird am Ende ein einzelner Parameterwert geschätzt:

$$\hat{\theta}_{\text{ML}} \quad \text{oder} \quad \hat{\theta}_{\text{MAP}}.$$

Bei Bayesscher Schätzung betrachtet man stattdessen die gesamte Posterior-Verteilung:

$$p(\theta \mid x_1, \dots, x_N).$$

Solution

Die Posterior-Verteilung beschreibt die Unsicherheit über den Parameter nach Beobachtung der Daten.

Bayes' Regel liefert:

$$p(\theta \mid x_1, \dots, x_N) = \frac{p(x_1, \dots, x_N \mid \theta)p(\theta)}{p(x_1, \dots, x_N)}.$$

Dabei ist:

$$p(x_1, \dots, x_N) = \int p(x_1, \dots, x_N \mid \theta)p(\theta) d\theta.$$

Diese Größe normalisiert den Posterior.

Answer

Bayessche Schätzung liefert nicht nur einen Punktwert, sondern eine ganze Verteilung:

$$p(\theta \mid x_1, \dots, x_N)$$

MAP wählt nur den wahrscheinlichsten Wert dieser Verteilung:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid x_1, \dots, x_N).$$

Der Bayessche Ansatz behält dagegen die Unsicherheit über θ bei.

3.16 Standard Derivation Technique for Maximum Likelihood Estimators

Notes

Für Maximum-Likelihood-Schätzer gibt es ein Standardvorgehen.

Calculation

Vorgehen zur Herleitung eines ML-Schätzers:

1. Modell bzw. Dichte aufschreiben:

$$p(x \mid \theta).$$

2. Likelihood für die Daten bilden:

$$L(\theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

3. Log-Likelihood bilden:

$$\ell(\theta) = \sum_{n=1}^N \log p(x_n \mid \theta).$$

4. Nach dem Parameter ableiten:

$$\frac{\partial}{\partial \theta} \ell(\theta).$$

5. Ableitung gleich null setzen:

$$\frac{\partial}{\partial \theta} \ell(\theta) = 0.$$

6. Nach θ auflösen.

Answer

Standardform für ML:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n \mid \theta)$$

Praktisch rechnet man meistens:

$$\frac{\partial}{\partial \theta} \ell(\theta) = 0$$

und löst nach θ auf.

3.17 Standard Derivation Technique for MAP Estimators

Notes

Für MAP-Schätzer ist die Herleitung fast gleich wie bei ML, aber zusätzlich kommt der Prior $p(\theta)$ hinzu.

Calculation

Vorgehen zur Herleitung eines MAP-Schätzers:

1. Modell bzw. Likelihood aufschreiben:

$$p(x_1, \dots, x_N \mid \theta).$$

2. Prior auf den Parameter aufschreiben:

$$p(\theta).$$

3. Posterior bis auf Normierung bilden:

$$p(\theta \mid x_1, \dots, x_N) \propto p(x_1, \dots, x_N \mid \theta)p(\theta).$$

4. Log-Posterior bilden:

$$\log p(\theta \mid x_1, \dots, x_N) = \sum_{n=1}^N \log p(x_n \mid \theta) + \log p(\theta) + \text{const.}$$

5. Nach θ ableiten und gleich null setzen.

Answer

Standardform für MAP:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{n=1}^N \log p(x_n \mid \theta) + \log p(\theta) \right]$$

Der Unterschied zu ML ist der zusätzliche Term:

$$\log p(\theta).$$

3.18 Prüfungsrelevante Kurzfassung

Answer

Lecture 3 behandelt Schätzer für unbekannte Parameter von Verteilungen.

Die wichtigsten Punkte sind:

- Für eine Gleichverteilung $X \sim \text{Unif}(a, b)$ gilt:

$$\mathbb{E}[X] = \frac{a+b}{2}, \quad \text{Var}(X) = \frac{(b-a)^2}{12}.$$

- Min-Max-Schätzer:

$$\hat{a} = x_{\min}, \quad \hat{b} = x_{\max}.$$

- Methode der Momente:

$$\hat{a}_{\text{MoM}} = \bar{x} - \sqrt{3v}, \quad \hat{b}_{\text{MoM}} = \bar{x} + \sqrt{3v}.$$

- i.i.d. bedeutet unabhängig und identisch verteilt:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n).$$

- Ein Schätzer ist eine Funktion der Daten:

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_N).$$

- Bias:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

- Varianz des Schätzers:

$$\text{Var}(\hat{\theta}) = \mathbb{E} \left[\left(\hat{\theta} - \mathbb{E}[\hat{\theta}] \right)^2 \right].$$

- Stichprobenmittelwert:

$$\bar{X} = \frac{1}{N} \sum_{n=1}^N X_n.$$

- Für den Stichprobenmittelwert gilt:

$$\mathbb{E}[\bar{X}] = \mu, \quad \text{Var}(\bar{X}) = \frac{\sigma^2}{N}.$$

- Unverzerrte Stichprobenvarianz:

$$S^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \bar{X})^2.$$

- Maximum Likelihood:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n | \theta).$$

- Maximum a-posteriori:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{n=1}^N \log p(x_n | \theta) + \log p(\theta) \right].$$

- Bayessche Schätzung betrachtet die ganze Posterior-Verteilung:

$$p(\theta \mid x_1, \dots, x_N).$$

Die Kernaussage lautet:

Lernen bedeutet oft, unbekannte Parameter aus Daten zu schätzen.

4 Lecture 4: Maximum Likelihood and Regression

4.1 Overview

Notes

Lecture 4 verbindet die Maximum-Likelihood-Schätzung aus Lecture 3 mit Regressions- und Klassifikationsmodellen.

Die zentralen Themen sind:

- Wiederholung von Schätzern und Maximum-Likelihood-Schätzern,
- Maximum-Likelihood-Schätzung für die Gleichverteilung,
- Maximum-Likelihood-Schätzung für Gauß-Verteilungen,
- Diskussion der Maximum-Likelihood-Methode,
- Regression mit gelabelten Daten,
- lineare Regression,
- Overfitting,
- Maximum-a-posteriori-Schätzung und Ridge Regression,
- Klassifikation als Regression,
- logistische Regression,
- Gradient Descent.

Answer

Die Kernaussage von Lecture 4 ist:

Viele Lernprobleme lassen sich als Parameterschätzung mit Maximum Likelihood oder MAP formulieren.

Regression und Klassifikation können dadurch probabilistisch interpretiert werden.

4.2 Recap: Estimators

Notes

Ein Schätzer ist eine Funktion der Daten, mit der ein unbekannter Parameter einer Verteilung geschätzt wird.

Sei θ ein unbekannter Parameter und seien

$$X_1, \dots, X_N$$

Zufallsvariablen.

Ein Schätzer ist dann:

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_N).$$

Solution

Wichtig ist, dass ein Schätzer selbst eine Zufallsvariable ist, weil er von zufälligen Daten abhängt.

Für konkrete beobachtete Daten

$$x_1, \dots, x_N$$

erhält man einen konkreten Wert:

$$\hat{\theta}(x_1, \dots, x_N).$$

Vor der Beobachtung der Daten ist $\hat{\theta}$ aber zufällig.

Answer

Ein Schätzer ist:

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_N)$$

Ein guter Schätzer sollte den wahren Parameter θ möglichst genau approximieren.

4.3 Recap: Maximum Likelihood Estimators

Notes

Die Maximum-Likelihood-Methode wählt den Parameter, unter dem die beobachteten Daten am wahrscheinlichsten sind.

Für Daten

$$x_1, \dots, x_N$$

und Parameter θ ist die Likelihood:

$$L(\theta) = p(x_1, \dots, x_N \mid \theta).$$

Calculation

Unter der i.i.d.-Annahme gilt:

$$p(x_1, \dots, x_N | \theta) = \prod_{n=1}^N p(x_n | \theta).$$

Der Maximum-Likelihood-Schätzer ist:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(x_1, \dots, x_N | \theta).$$

Für i.i.d. Daten:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{n=1}^N p(x_n | \theta).$$

Calculation

Meistens verwendet man die Log-Likelihood:

$$\ell(\theta) = \log L(\theta).$$

Da der Logarithmus streng monoton wachsend ist, gilt:

$$\arg \max_{\theta} L(\theta) = \arg \max_{\theta} \ell(\theta).$$

Für i.i.d. Daten folgt:

$$\ell(\theta) = \sum_{n=1}^N \log p(x_n | \theta).$$

Answer

Maximum Likelihood:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{n=1}^N p(x_n | \theta)$$

Äquivalent in Log-Form:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n | \theta)$$

4.4 Uniform Distribution as Maximum Likelihood Estimation

Notes

Wir betrachten Daten

$$x_1, \dots, x_N$$

aus einer Gleichverteilung:

$$X \sim \text{Unif}(a, b).$$

Die Dichte lautet:

$$p(x \mid a, b) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b, \\ 0, & \text{sonst.} \end{cases}$$

Calculation

Für i.i.d. Daten ist die Likelihood:

$$L(a, b) = \prod_{n=1}^N p(x_n \mid a, b).$$

Wenn alle Datenpunkte im Intervall $[a, b]$ liegen, gilt:

$$L(a, b) = \prod_{n=1}^N \frac{1}{b-a} = \frac{1}{(b-a)^N}.$$

Wenn mindestens ein Datenpunkt außerhalb von $[a, b]$ liegt, ist:

$$L(a, b) = 0.$$

Calculation

Um die Likelihood zu maximieren, muss das Intervall $[a, b]$ alle Daten enthalten. Gleichzeitig soll $b - a$ möglichst klein sein, da

$$L(a, b) = \frac{1}{(b-a)^N}$$

größer wird, wenn $b - a$ kleiner wird.

Das kleinste Intervall, das alle Daten enthält, ist:

$$a = x_{\min}$$

und

$$b = x_{\max}.$$

Answer

Für die Gleichverteilung liefert Maximum Likelihood:

$$\hat{a}_{\text{ML}} = x_{\min}$$

$$\hat{b}_{\text{ML}} = x_{\max}$$

Das entspricht der Min-Max-Methode aus Lecture 3.

4.5 Testing Maximum Likelihood Estimators for Gaussian Distributions

Notes

Ein wichtiges Testbeispiel für Maximum Likelihood ist die Normalverteilung.
Sei:

$$X \sim \mathcal{N}(\mu, \sigma^2).$$

Die Dichte lautet:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Calculation

Für i.i.d. Daten $x_1, \dots, x_N$ ist die Likelihood:

$$L(\mu, \sigma^2) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right).$$

Die Log-Likelihood ist:

$$\ell(\mu, \sigma^2) = -\frac{N}{2} \log(2\pi) - \frac{N}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2.$$

Calculation

Ableitung nach μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{n=1}^N (x_n - \mu).$$

Setze gleich null:

$$\frac{1}{\sigma^2} \sum_{n=1}^N (x_n - \mu) = 0.$$

Also:

$$\sum_{n=1}^N x_n - N\mu = 0.$$

Damit:

$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n = \bar{x}.$$

Calculation

Ableitung nach σ^2 liefert:

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \hat{\mu}_{\text{ML}})^2.$$

Dieser Schätzer ist der ML-Schätzer für die Varianz, aber er ist im Allgemeinen verzerrt. Der unverzernte Schätzer verwendet stattdessen:

$$\frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})^2.$$

Answer

Für eine Gauß-Verteilung gilt:

$$\hat{\mu}_{\text{ML}} = \bar{x} = \frac{1}{N} \sum_{n=1}^N x_n$$

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})^2$$

Der ML-Varianzschätzer verwendet $1/N$, nicht $1/(N-1)$.

4.6 Discussion of the Maximum Likelihood Method

Notes

Maximum Likelihood ist eine sehr allgemeine Methode zur Parameterschätzung. Man braucht dafür:

- ein probabilistisches Modell $p(x \mid \theta)$,
- beobachtete Daten $x_1, \dots, x_N$,
- eine Optimierung der Likelihood oder Log-Likelihood.

Solution

Vorteile von Maximum Likelihood:

- systematische Methode,
- auf viele Modelle anwendbar,

- oft rechnerisch gut handhabbar über die Log-Likelihood,
- direkte probabilistische Interpretation.

Mögliche Nachteile:

- kann bei kleinen Datenmengen stark schwanken,
- nutzt kein Vorwissen über Parameter,
- kann zu Overfitting führen,
- Optimierungsproblem kann schwierig sein.

Answer

Maximum Likelihood bedeutet:

Wähle den Parameter, unter dem die beobachteten Daten maximal plausibel sind.

MAP erweitert diese Idee um Vorwissen durch einen Prior:

$$p(\theta).$$

4.7 Regression: Problem Definition and Maximum Likelihood for Labeled Data

Notes

Bei Regression möchte man aus Eingaben x kontinuierliche Zielwerte t vorhersagen.
Die Trainingsdaten sind gelabelte Paare:

$$\mathcal{D} = \{(x_1, t_1), \dots, (x_N, t_N)\}.$$

Dabei ist:

$$x_n = \text{Input}$$

und

$$t_n = \text{Target bzw. Label.}$$

Solution

In einem probabilistischen Regressionsmodell beschreibt man nicht nur eine deterministische Funktion

$$y(x, w),$$

sondern eine Verteilung über Zielwerte:

$$p(t \mid x, w).$$

Der Parametervektor w bestimmt das Modell.

Für gelabelte Daten lautet die Likelihood:

$$L(w) = p(t_1, \dots, t_N \mid x_1, \dots, x_N, w).$$

Unter der Annahme unabhängiger Daten:

$$L(w) = \prod_{n=1}^N p(t_n \mid x_n, w).$$

Answer

Regression mit Maximum Likelihood:

$$\hat{w}_{\text{ML}} = \arg \max_w \prod_{n=1}^N p(t_n \mid x_n, w)$$

In Log-Form:

$$\hat{w}_{\text{ML}} = \arg \max_w \sum_{n=1}^N \log p(t_n \mid x_n, w)$$

4.8 Examples of Regression Problems

Notes

Regression wird verwendet, wenn die Zielvariable kontinuierlich ist.

Beispiele:

- Vorhersage von Hauspreisen,
- Vorhersage von Temperatur,
- Vorhersage von Blutdruck oder medizinischen Messwerten,
- Vorhersage von Verkaufszahlen,
- Schätzung einer physikalischen Größe aus Messdaten.

Answer

Regression bedeutet:

$$x \mapsto t$$

wobei t ein kontinuierlicher Zielwert ist.

Im Unterschied zur Klassifikation ist das Ziel nicht eine Klasse, sondern ein numerischer Wert.

4.9 Linear Regression: Definition

Notes

Bei linearer Regression wird der Zielwert durch eine lineare Funktion der Eingabe beschrieben.

Für eine eindimensionale Eingabe:

$$y(x, w) = w_0 + w_1 x.$$

Allgemeiner mit Basisfunktionen $\phi_j(x)$:

$$y(x, w) = \sum_{j=0}^M w_j \phi_j(x).$$

In Vektorform:

$$y(x, w) = w^T \phi(x).$$

Solution

Die Funktion ist linear in den Parametern w_j , auch wenn die Basisfunktionen $\phi_j(x)$ nicht-linear in x sein können.

Beispiele für Basisfunktionen:

$$\phi_0(x) = 1, \quad \phi_1(x) = x, \quad \phi_2(x) = x^2.$$

Dann ist:

$$y(x, w) = w_0 + w_1 x + w_2 x^2.$$

Dieses Modell ist nichtlinear in x , aber linear in den Parametern.

Answer

Lineare Regression verwendet Modelle der Form:

$$y(x, w) = w^T \phi(x)$$

Wichtig:

linear in den Parametern w .

4.10 Linear Regression via Maximum Likelihood

Notes

Für die ML-Herleitung der linearen Regression nimmt man meist ein Gaußsches Rauschmodell an.

Das Modell lautet:

$$t_n = y(x_n, w) + \varepsilon_n,$$

wobei

$$\varepsilon_n \sim \mathcal{N}(0, \sigma^2).$$

Daher:

$$p(t_n \mid x_n, w, \sigma^2) = \mathcal{N}(t_n \mid y(x_n, w), \sigma^2).$$

Calculation

Für unabhängige Daten ist die Likelihood:

$$L(w) = \prod_{n=1}^N \mathcal{N}(t_n \mid y(x_n, w), \sigma^2).$$

Die Log-Likelihood ist bis auf Konstanten:

$$\ell(w) = -\frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - y(x_n, w))^2 + \text{const.}$$

Maximierung der Log-Likelihood ist daher äquivalent zur Minimierung der Summe quadratischer Fehler:

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2.$$

Answer

Lineare Regression mit Gaußschem Rauschen führt zu Least Squares:

$$\hat{w}_{\text{ML}} = \arg \min_w \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2$$

Das bedeutet:

Least Squares ist Maximum Likelihood unter Gaußschem Rauschen.

4.11 Overfitting

Notes

Overfitting tritt auf, wenn ein Modell die Trainingsdaten sehr gut erklärt, aber schlecht auf neue Daten generalisiert.

Typisch ist:

Training Error klein, Test Error groß.

Solution

Overfitting entsteht häufig, wenn das Modell zu flexibel ist, also zu viele Parameter oder zu komplexe Basisfunktionen verwendet.

Dann kann das Modell nicht nur die zugrundeliegende Struktur lernen, sondern auch Rauschen in den Trainingsdaten.

In Regression kann dies zum Beispiel passieren, wenn ein Polynom sehr hohen Grades verwendet wird.

Answer

Overfitting bedeutet:

Das Modell passt die Trainingsdaten zu stark an und generalisiert schlecht.

Gegenmaßnahmen sind zum Beispiel:

- Regularisierung,
- einfachere Modelle,
- mehr Daten,
- Validierungsdaten zur Modellwahl.

4.12 Linear Regression via MAP: Ridge Regression

Notes

MAP-Schätzung erweitert Maximum Likelihood um einen Prior auf die Parameter.

Für lineare Regression verwendet man häufig einen Gauß-Prior:

$$p(w) = \mathcal{N}(w \mid 0, \alpha^{-1} I).$$

Dieser Prior bevorzugt kleine Gewichte.

Calculation

Die MAP-Schätzung maximiert:

$$p(w \mid \mathcal{D}) \propto p(\mathcal{D} \mid w)p(w).$$

In Log-Form:

$$\log p(w \mid \mathcal{D}) = \log p(\mathcal{D} \mid w) + \log p(w) + \text{const.}$$

Der Likelihood-Term mit Gaußischem Rauschen führt zu:

$$-\frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - y(x_n, w))^2.$$

Der Gauß-Prior führt zu:

$$-\frac{\alpha}{2} w^T w.$$

Calculation

Maximierung des Log-Posteriors ist äquivalent zur Minimierung:

$$E_{\text{ridge}}(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2 + \frac{\lambda}{2} w^T w.$$

Dabei ist λ ein Regularisierungsparameter.

Der zusätzliche Term

$$\frac{\lambda}{2} w^T w$$

bestraft große Gewichte.

Answer

Ridge Regression ergibt sich als MAP-Schätzung mit Gauß-Prior auf w :

$$\hat{w}_{\text{MAP}} = \arg \min_w \left[\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2 + \frac{\lambda}{2} w^T w \right]$$

Der Regularisierungsterm reduziert Overfitting.

4.13 Classification as Regression

Notes

Klassifikation kann teilweise als Regressionsproblem formuliert werden.

Statt einen kontinuierlichen Zielwert vorherzusagen, soll das Modell eine Klassenwahrscheinlichkeit ausgeben.

Für binäre Klassifikation:

$$t \in \{0, 1\}.$$

Gesucht ist:

$$P(t = 1 \mid x).$$

Solution

Ein lineares Modell

$$y(x, w) = w^T x$$

kann beliebige reelle Werte annehmen.

Für Wahrscheinlichkeiten brauchen wir aber Werte im Intervall:

$$[0, 1].$$

Daher verwendet man eine nichtlineare Funktion, die reelle Zahlen auf Wahrscheinlichkeiten abbildet.

Bei logistischer Regression ist das die Sigmoidfunktion.

Answer

Klassifikation als Regression bedeutet:

Wir modellieren eine Klassenwahrscheinlichkeit als Funktion von x .

Für binäre Klassifikation:

$$P(t = 1 \mid x, w) \in [0, 1].$$

4.14 Logistic Regression: Definition

Notes

Logistische Regression ist ein Modell für binäre Klassifikation.

Die Zielvariable ist:

$$t \in \{0, 1\}.$$

Das Modell beschreibt:

$$P(t = 1 \mid x, w).$$

Calculation

Zuerst bildet man eine lineare Kombination:

$$a = w^T x.$$

Dann wird diese durch die Sigmoidfunktion transformiert:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Damit:

$$P(t = 1 \mid x, w) = \sigma(w^T x).$$

Und:

$$P(t = 0 \mid x, w) = 1 - \sigma(w^T x).$$

Answer

Logistische Regression:

$$P(t = 1 \mid x, w) = \sigma(w^T x)$$

mit

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

Die Sigmoidfunktion macht aus einem reellen Wert eine Wahrscheinlichkeit.

4.15 Excursion: Gradient Descent

Notes

Viele Modelle haben keine geschlossene Lösung für die optimalen Parameter. Dann verwendet man iterative Optimierungsverfahren wie Gradient Descent. Ziel ist es, eine Fehlerfunktion $E(w)$ zu minimieren.

Calculation

Gradient Descent aktualisiert die Parameter in Richtung des negativen Gradienten:

$$w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E(w^{(\tau)}).$$

Dabei ist:

- τ der Iterationsindex,
- $\eta > 0$ die Lernrate,
- $\nabla E(w)$ der Gradient der Fehlerfunktion.

Solution

Der Gradient zeigt in Richtung des stärksten Anstiegs der Funktion.
Da man minimieren möchte, geht man in die entgegengesetzte Richtung:

$$-\nabla E(w).$$

Die Lernrate η bestimmt, wie groß jeder Schritt ist.
Ist η zu groß, kann das Verfahren instabil werden.
Ist η zu klein, konvergiert das Verfahren langsam.

Answer

Gradient Descent:

$$w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E(w^{(\tau)})$$

Es ist ein iteratives Verfahren zur Minimierung einer Fehlerfunktion.

4.16 Logistic Regression via Maximum Likelihood

Notes

Logistische Regression kann mit Maximum Likelihood hergeleitet werden.
Für binäre Labels

$$t_n \in \{0,1\}$$

modelliert man:

$$P(t_n = 1 \mid x_n, w) = y_n = \sigma(w^T x_n).$$

Calculation

Die Bernoulli-Wahrscheinlichkeit für ein Label t_n ist:

$$p(t_n \mid x_n, w) = y_n^{t_n} (1 - y_n)^{1-t_n}.$$

Für i.i.d. Daten ist die Likelihood:

$$L(w) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}.$$

Die Log-Likelihood ist:

$$\ell(w) = \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Calculation

Maximum Likelihood maximiert $\ell(w)$.

Äquivalent minimiert man die negative Log-Likelihood:

$$E(w) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Diese Fehlerfunktion heißt Cross-Entropy Loss.

Answer

Logistische Regression mit Maximum Likelihood führt zur Cross-Entropy:

$$E(w) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)]$$

mit

$$y_n = \sigma(w^T x_n).$$

Dieser Fehler wird typischerweise mit Gradient Descent minimiert.

4.17 Summary

Answer

Lecture 4 zeigt, wie Maximum Likelihood und MAP-Schätzung zu konkreten Machine-Learning-Modellen führen.

Die wichtigsten Punkte sind:

- Maximum Likelihood wählt Parameter, die die Daten am plausibelsten machen:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n | \theta).$$

- Für eine Gleichverteilung liefert ML:

$$\hat{a}_{\text{ML}} = x_{\min}, \quad \hat{b}_{\text{ML}} = x_{\max}.$$

- Für eine Gauß-Verteilung liefert ML:

$$\hat{\mu}_{\text{ML}} = \bar{x}, \quad \hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})^2.$$

- Regression verwendet gelabelte Daten:

$$(x_n, t_n).$$

- Lineare Regression modelliert:

$$y(x, w) = w^T \phi(x).$$

- Lineare Regression mit Gaußschem Rauschen führt zu Least Squares:

$$\hat{w}_{\text{ML}} = \arg \min_w \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2.$$

- MAP mit Gauß-Prior auf w führt zu Ridge Regression:

$$\hat{w}_{\text{MAP}} = \arg \min_w \left[\frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2 + \frac{\lambda}{2} w^T w \right].$$

- Logistische Regression modelliert:

$$P(t = 1 \mid x, w) = \sigma(w^T x).$$

- Die Sigmoidfunktion ist:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

- Logistische Regression mit ML führt zur Cross-Entropy:

$$E(w) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

- Gradient Descent aktualisiert Parameter durch:

$$w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E(w^{(\tau)}).$$

Die Kernaussage lautet:

Regression und Klassifikation können als probabilistische Parameterschätzung formuliert werden.

5 Lecture 5: Neural Networks – Perceptrons and Multi-Layer Perceptrons

5.1 Overview

Notes

Lecture 5 führt neuronale Netze ein. Der erste Teil behandelt einfache Perzeptrons, der zweite Teil erweitert diese Idee zu Multi-Layer Perceptrons.

Die zentralen Themen sind:

- biologische und künstliche neuronale Netze,
- Definition des Perzeptrons,
- Lernen in Perzeptrons,
- Perzeptrons als Klassifikatoren und ihre Grenzen,
- das XOR-Problem,
- Erweiterung zu Multi-Layer Perceptrons,
- Herleitung von Lernregeln für MLPs,
- mechanistische Sicht auf Lernen durch Backpropagation.

Answer

Die Kernaussage von Lecture 5 ist:

Neuronale Netze lernen Funktionen durch gewichtete Kombinationen, Nichtlinearitäten und schrittweise A

Ein einzelnes Perzeptron kann nur linear separierbare Probleme lösen. Multi-Layer Perceptrons erweitern diese Idee und können deutlich komplexere Funktionen darstellen.

5.2 Part A: Biological and Artificial Neural Networks

Notes

Die Motivation für künstliche neuronale Netze stammt historisch aus biologischen Nervensystemen.

Ein biologisches Neuron erhält Eingangssignale über Dendriten, verarbeitet diese im Zellkörper und sendet bei ausreichender Aktivierung ein Signal über das Axon weiter.

Die grobe Analogie in künstlichen neuronalen Netzen ist:

- Eingänge entsprechen Eingangssignalen,
- Gewichte entsprechen der Stärke von Verbindungen,

- eine gewichtete Summe entspricht der Integration von Signalen,
- eine Aktivierungsfunktion entscheidet über die Ausgabe.

Solution

Ein künstliches Neuron erhält Eingaben

$$x_1, \dots, x_D.$$

Jede Eingabe hat ein Gewicht

$$w_1, \dots, w_D.$$

Zusätzlich gibt es oft einen Bias-Term b . Zuerst wird eine gewichtete Summe gebildet:

$$a = \sum_{j=1}^D w_j x_j + b.$$

Danach wird diese Aktivierung durch eine Funktion g transformiert:

$$y = g(a).$$

Das ist die Grundstruktur eines künstlichen Neurons.

Answer

Ein künstliches Neuron berechnet:

$$a = \sum_{j=1}^D w_j x_j + b$$

und anschließend:

$$y = g(a)$$

Dabei sind w_j die Gewichte, b der Bias und g die Aktivierungsfunktion.

5.3 Definition of the Perceptron

Notes

Das Perzeptron ist eines der einfachsten künstlichen neuronalen Modelle.

Es ist ein binärer Klassifikator. Das bedeutet: Es entscheidet zwischen zwei Klassen.

Die Eingabe ist ein Vektor:

$$x = (x_1, \dots, x_D)^T.$$

Das Perzeptron besitzt Gewichte:

$$w = (w_1, \dots, w_D)^T$$

und einen Bias b .

Calculation

Zuerst berechnet das Perzeptron die Aktivierung:

$$a = w^T x + b.$$

Dann wird eine Schwellenfunktion angewendet.

Eine typische Definition ist:

$$y = \begin{cases} 1, & w^T x + b \geq 0, \\ 0, & w^T x + b < 0. \end{cases}$$

Alternativ verwendet man Labels $\{-1, +1\}$:

$$y = \text{sign}(w^T x + b).$$

Solution

Die Entscheidungsgrenze des Perzeptrons ist durch

$$w^T x + b = 0$$

gegeben.

In zwei Dimensionen ist das eine Gerade.

In höheren Dimensionen ist es eine Hyperebene.

Das Perzeptron klassifiziert danach, auf welcher Seite dieser Hyperebene ein Datenpunkt liegt.

Answer

Das Perzeptron ist definiert durch:

$$a = w^T x + b$$

und

$$y = \begin{cases} 1, & a \geq 0, \\ 0, & a < 0. \end{cases}$$

Die Entscheidungsgrenze ist:

$$w^T x + b = 0.$$

5.4 Learning in Perceptrons

Notes

Lernen bedeutet beim Perzeptron, die Gewichte w und den Bias b so anzupassen, dass die Trainingsdaten möglichst korrekt klassifiziert werden.

Die Trainingsdaten sind gelabelte Beispiele:

$$\mathcal{D} = \left\{ (x^{(1)}, t^{(1)}), \dots, (x^{(N)}, t^{(N)}) \right\}.$$

Dabei ist $t^{(n)}$ das gewünschte Label.

Solution

Das Perzeptron macht für einen Datenpunkt $x^{(n)}$ eine Vorhersage:

$$y^{(n)} = f(w^T x^{(n)} + b).$$

Wenn die Vorhersage korrekt ist, müssen die Gewichte nicht verändert werden.

Wenn die Vorhersage falsch ist, werden die Gewichte so angepasst, dass die Entscheidung beim nächsten Mal eher in Richtung des richtigen Labels geht.

Calculation

Für Labels $t \in \{-1, +1\}$ und Perzeptron-Ausgabe

$$y = \text{sign}(w^T x + b)$$

kann die Perzeptron-Lernregel geschrieben werden als:

$$w_{\text{new}} = w_{\text{old}} + \eta t x$$

falls der Punkt falsch klassifiziert wurde.

Für den Bias:

$$b_{\text{new}} = b_{\text{old}} + \eta t.$$

Dabei ist $\eta > 0$ die Lernrate.

Side Calculation

Warum macht diese Regel Sinn?

Wenn $t = +1$, der Punkt aber falsch klassifiziert wurde, dann ist

$$w^T x + b < 0.$$

Durch

$$w \leftarrow w + \eta x$$

wird die Aktivierung

$$w^T x + b$$

für diesen Punkt größer.

Wenn $t = -1$, der Punkt aber falsch klassifiziert wurde, dann ist

$$w^T x + b > 0.$$

Durch

$$w \leftarrow w - \eta x$$

wird die Aktivierung kleiner.

Answer

Die Perzeptron-Lernregel für falsch klassifizierte Punkte lautet:

$$w \leftarrow w + \eta t x$$

$$b \leftarrow b + \eta t$$

mit Labels $t \in \{-1, +1\}$.

Die Regel verändert die Entscheidungsgrenze so, dass der falsch klassifizierte Punkt eher auf die richtige Seite verschoben wird.

5.5 Perceptrons as Classifiers

Notes

Ein Perzeptron ist ein linearer Klassifikator.

Das bedeutet: Es trennt Klassen durch eine lineare Entscheidungsgrenze.

In zwei Dimensionen:

$$w_1 x_1 + w_2 x_2 + b = 0$$

ist eine Gerade.

In D Dimensionen:

$$w^T x + b = 0$$

ist eine Hyperebene.

Solution

Ein Datensatz heißt linear separierbar, wenn es eine Gerade beziehungsweise Hyperebene gibt, die die Klassen vollständig trennt.

Für linear separierbare Daten kann ein Perzeptron prinzipiell eine passende Entscheidungsgrenze lernen.

Wenn die Daten nicht linear separierbar sind, reicht ein einzelnes Perzeptron nicht aus.

Answer

Das Perzeptron ist ein linearer Klassifikator:

$$y = \text{sign}(w^T x + b)$$

Es kann nur Entscheidungsgrenzen der Form

$$w^T x + b = 0$$

darstellen.

Daher kann es nur linear separierbare Klassifikationsprobleme lösen.

5.6 Limitations of Perceptrons: The XOR Problem

Notes

Das klassische Beispiel für die Grenzen eines einzelnen Perzeptrons ist das XOR-Problem. Die XOR-Funktion ist definiert durch:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1,$$

$$1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

Die positiven Beispiele liegen also diagonal gegenüber, ebenso die negativen.

Solution

Die vier Punkte sind:

$$(0,0) \mapsto 0, \quad (1,0) \mapsto 1,$$

$$(0,1) \mapsto 1, \quad (1,1) \mapsto 0.$$

Diese Punkte können nicht durch eine einzige Gerade getrennt werden.

Daher kann ein einzelnes Perzeptron XOR nicht darstellen.

Answer

Das XOR-Problem zeigt:

Ein einzelnes Perzeptron kann nur linear separierbare Probleme lösen.

Da XOR nicht linear separierbar ist, gilt:

Ein einzelnes Perzeptron kann XOR nicht lösen.

Dafür braucht man mehrere Schichten beziehungsweise nichtlineare Zwischenrepräsentationen.

5.7 Part B: Extension to Multi-Layer Perceptrons

Notes

Ein Multi-Layer Perceptron, kurz MLP, erweitert das einfache Perzeptron durch mehrere Schichten.

Typische Schichten sind:

- Eingabeschicht,
- eine oder mehrere versteckte Schichten,
- Ausgabeschicht.

Die versteckten Schichten erlauben es, nichtlineare Zwischenrepräsentationen zu lernen.

Solution

Eine Schicht eines neuronalen Netzes berechnet typischerweise:

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$$

und danach:

$$h^{(\ell)} = g(a^{(\ell)}).$$

Dabei ist:

- ℓ der Schichtindex,
- $W^{(\ell)}$ die Gewichtsmatrix der Schicht,
- $b^{(\ell)}$ der Bias-Vektor,
- $h^{(\ell-1)}$ die Eingabe der Schicht,
- $h^{(\ell)}$ die Ausgabe der Schicht,
- g eine nichtlineare Aktivierungsfunktion.

Answer

Eine MLP-Schicht hat die Form:

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$$

$$h^{(\ell)} = g(a^{(\ell)})$$

Durch mehrere solche Schichten entstehen komplexe nichtlineare Funktionen.

5.8 Why Nonlinear Activation Functions Are Necessary

Notes

Die Nichtlinearität in den Aktivierungsfunktionen ist entscheidend.

Wenn alle Aktivierungsfunktionen linear wären, dann wäre die Verkettung mehrerer Schichten wieder nur eine lineare Funktion.

Calculation

Betrachte zwei lineare Schichten ohne Nichtlinearität:

$$h^{(1)} = W^{(1)}x + b^{(1)}$$

und

$$y = W^{(2)}h^{(1)} + b^{(2)}.$$

Einsetzen liefert:

$$y = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}.$$

Also:

$$y = W^{(2)}W^{(1)}x + W^{(2)}b^{(1)} + b^{(2)}.$$

Das ist wieder von der Form:

$$y = \widetilde{W}x + \widetilde{b}.$$

Also wieder linear.

Answer

Ohne Nichtlinearitäten gilt:

Mehrere lineare Schichten ergeben zusammen wieder nur eine lineare Schicht.

Deshalb braucht ein MLP nichtlineare Aktivierungsfunktionen, zum Beispiel Sigmoid, tanh oder ReLU.

5.9 Learning Rules for Multi-Layer Perceptrons

Notes

Beim Lernen eines MLPs sollen alle Gewichte und Biases so angepasst werden, dass eine Fehlerfunktion minimiert wird.

Sei die Fehlerfunktion:

$$E(\theta),$$

wobei θ alle Parameter des Netzes zusammenfasst.

Die Parameter sind:

$$\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots\}.$$

Solution

Das Standardprinzip ist Gradient Descent.

Für einen Parameter θ_i lautet die Update-Regel:

$$\theta_i \leftarrow \theta_i - \eta \frac{\partial E}{\partial \theta_i}.$$

Für Gewichtsmatrizen schreibt man entsprechend:

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial E}{\partial W^{(\ell)}}.$$

Für Biases:

$$b^{(\ell)} \leftarrow b^{(\ell)} - \eta \frac{\partial E}{\partial b^{(\ell)}}.$$

Answer

Lernen in MLPs bedeutet Gradientenabstieg auf einer Fehlerfunktion:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} E(\theta)$$

Für jede Schicht:

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial E}{\partial W^{(\ell)}}$$

$$b^{(\ell)} \leftarrow b^{(\ell)} - \eta \frac{\partial E}{\partial b^{(\ell)}}$$

5.10 Mechanistic View of Learning: Backpropagation

Notes

Backpropagation ist der Algorithmus, mit dem Gradienten in mehrschichtigen neuronalen Netzen effizient berechnet werden.

Die zentrale Idee ist die Kettenregel der Differentialrechnung.

Da ein MLP eine Verkettung vieler Funktionen ist, kann man die Ableitungen von hinten nach vorne durch das Netzwerk propagieren.

Solution

Der Lernprozess besteht aus zwei Hauptphasen:

1. **Forward pass:** Die Eingabe wird Schicht für Schicht durch das Netzwerk gerechnet, bis eine Ausgabe entsteht.
2. **Backward pass:** Der Fehler wird von der Ausgabeschicht zurück durch das Netzwerk propagiert, um die Gradienten aller Gewichte zu berechnen.

Calculation

Für eine Schicht gilt:

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$$

und

$$h^{(\ell)} = g(a^{(\ell)}).$$

Um die Gewichte zu aktualisieren, benötigt man:

$$\frac{\partial E}{\partial W^{(\ell)}}.$$

Mit der Kettenregel zerlegt man diese Ableitung in lokale Ableitungen:

$$\frac{\partial E}{\partial W^{(\ell)}} = \frac{\partial E}{\partial a^{(\ell)}} \frac{\partial a^{(\ell)}}{\partial W^{(\ell)}}.$$

Definiert man den Fehlerterm

$$\delta^{(\ell)} = \frac{\partial E}{\partial a^{(\ell)}},$$

dann gilt für die Gewichtgradienten:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T.$$

Calculation

Für die Biases gilt entsprechend:

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Die Fehlerterme $\delta^{(\ell)}$ werden rückwärts berechnet.

Für eine versteckte Schicht gilt schematisch:

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot g'(a^{(\ell)}).$$

Dabei bezeichnet $\odot$ die komponentenweise Multiplikation.

Answer

Backpropagation besteht aus:

- Forward pass: Ausgaben berechnen.
- Loss berechnen.
- Backward pass: Gradienten mit der Kettenregel berechnen.
- Parameter mit Gradient Descent aktualisieren.

Zentraler Fehlerterm:

$$\delta^{(\ell)} = \frac{\partial E}{\partial a^{(\ell)}}$$

Gewichtgradient:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T$$

Hidden-layer-Rekursion:

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot g'(a^{(\ell)})$$

5.11 Perceptrons, MLPs and the XOR Problem

Notes

Das XOR-Problem motiviert den Übergang vom einzelnen Perzeptron zum MLP.

Ein einzelnes Perzeptron kann nur eine lineare Entscheidungsgrenze bilden.

XOR benötigt aber eine nichtlineare Trennung.

Solution

Ein MLP mit mindestens einer versteckten Schicht kann zuerst eine neue Repräsentation der Eingabe lernen.

In dieser versteckten Repräsentation kann ein Problem linear separierbar werden, obwohl es im ursprünglichen Eingaberaum nicht linear separierbar war.

Genau dadurch können MLPs Probleme wie XOR lösen.

Answer

Der Übergang ist:

Perzeptron = linearer Klassifikator

MLP = mehrschichtiges nichtlineares Modell

Ein MLP kann durch versteckte Schichten komplexere Entscheidungsgrenzen darstellen.

5.12 Prüfungsrelevante Kurzfassung

Answer

Lecture 5 behandelt Perzeptrons und Multi-Layer Perceptrons.

Die wichtigsten Punkte sind:

- Ein künstliches Neuron berechnet:

$$a = w^T x + b, \quad y = g(a).$$

- Ein Perzeptron ist ein binärer linearer Klassifikator:

$$y = \text{sign}(w^T x + b).$$

- Die Entscheidungsgrenze ist:

$$w^T x + b = 0.$$

- Für falsch klassifizierte Punkte lautet die Perzeptron-Lernregel:

$$w \leftarrow w + \eta tx, \quad b \leftarrow b + \eta t.$$

- Ein einzelnes Perzeptron kann nur linear separierbare Probleme lösen.
- XOR ist nicht linear separierbar und kann daher nicht durch ein einzelnes Perzeptron gelöst werden.

- Ein MLP besteht aus mehreren Schichten:

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = g(a^{(\ell)}).$$

- Nichtlineare Aktivierungsfunktionen sind notwendig, weil mehrere lineare Schichten zusammen wieder nur eine lineare Funktion ergeben.
- Lernen in MLPs erfolgt durch Gradient Descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} E(\theta).$$

- Backpropagation berechnet die Gradienten effizient mit der Kettenregel.
- Der Fehlerterm einer Schicht ist:

$$\delta^{(\ell)} = \frac{\partial E}{\partial a^{(\ell)}}.$$

- Für Gewichtsmatrizen gilt:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T.$$

Die Kernaussage lautet:

MLPs erweitern Perzeptrons durch versteckte Schichten und Nichtlinearitäten, sodass komplexe Funktionen

6 Lecture 6: Neural Networks – DNNs and Their Capabilities

6.1 Overview

Notes

Lecture 6 erweitert die Diskussion zu Multi-Layer Perceptrons und betrachtet, warum tiefe neuronale Netze leistungsfähig sind.

Die zentralen Themen sind:

- Verallgemeinerung der Backpropagation-Herleitung,
- Eigenschaften von Multi-Layer Perceptrons,
- MLPs und das XOR-Problem,
- Overfitting und Generalization Error bei MLPs,
- Validation Sets für MLPs,
- MLPs und Maximum Likelihood Estimation,

- Generalisierung und Validierung,
- andere neuronale Netzwerkarchitekturen.

Answer

Die Kernaussage von Lecture 6 ist:

Tiefe neuronale Netze lernen komplexe Funktionen durch Komposition vieler einfacher nichtlinearer Transf

Ihre Leistungsfähigkeit hängt aber nicht nur von der Modellklasse ab, sondern auch von Training, Regularisierung, Datenmenge und Validierung.

6.2 Part C.1: Generalization of Backpropagation

Notes

Backpropagation ist eine systematische Anwendung der Kettenregel auf mehrschichtige neuronale Netze.

Ein MLP besteht aus mehreren Schichten der Form

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$$

und

$$h^{(\ell)} = g^{(\ell)}(a^{(\ell)}).$$

Dabei ist:

$$h^{(0)} = x$$

die Eingabe des Netzes.

Solution

Der Forward Pass berechnet Schicht für Schicht:

$$x = h^{(0)} \longrightarrow h^{(1)} \longrightarrow h^{(2)} \longrightarrow \dots \longrightarrow h^{(L)}.$$

Die letzte Ausgabe $h^{(L)}$ wird mit dem Zielwert t verglichen. Daraus entsteht eine Fehlerfunktion:

$$E = E(h^{(L)}, t).$$

Beim Backward Pass werden dann die Ableitungen dieser Fehlerfunktion nach allen Parametern berechnet.

Answer

Ein MLP ist eine Komposition von Funktionen:

$$h^{(L)} = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x)$$

Backpropagation berechnet effizient:

$$\nabla_{\theta} E$$

für alle Parameter θ des Netzwerks.

6.3 Generalization of the Backpropagation Derivation

Notes

Die Backpropagation-Herleitung aus Lecture 5 lässt sich auf beliebig viele Schichten verallgemeinern.

Die zentrale Größe ist der Fehlerterm einer Schicht:

$$\delta^{(\ell)} = \frac{\partial E}{\partial a^{(\ell)}}.$$

Dieser Fehlerterm beschreibt, wie stark eine Änderung der Voraktivierung $a^{(\ell)}$ den Gesamtfehler beeinflusst.

Calculation

Für die letzte Schicht L gilt allgemein:

$$\delta^{(L)} = \frac{\partial E}{\partial a^{(L)}}.$$

Falls

$$h^{(L)} = g^{(L)}(a^{(L)}),$$

dann erhält man mit der Kettenregel:

$$\delta^{(L)} = \frac{\partial E}{\partial h^{(L)}} \odot g'^{(L)}(a^{(L)}).$$

Dabei bezeichnet $\odot$ komponentenweise Multiplikation.

Calculation

Für eine versteckte Schicht ℓ hängt der Fehler indirekt über die nächste Schicht von $a^{(\ell)}$ ab.

Die Rekursion lautet:

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot g'^{(\ell)}(a^{(\ell)}).$$

Damit können die Fehlerterme rückwärts berechnet werden:

$$\delta^{(L)} \longrightarrow \delta^{(L-1)} \longrightarrow \dots \longrightarrow \delta^{(1)}.$$

Calculation

Sind die Fehlerterme $\delta^{(\ell)}$ bekannt, erhält man die Gradienten:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T.$$

Für den Bias gilt:

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Die Parameter werden dann mit Gradient Descent aktualisiert:

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial E}{\partial W^{(\ell)}}$$

und

$$b^{(\ell)} \leftarrow b^{(\ell)} - \eta \frac{\partial E}{\partial b^{(\ell)}}.$$

Answer

Die allgemeine Backpropagation-Rekursion lautet:

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot g'^{(\ell)}(a^{(\ell)})$$

und die Gradienten sind:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T$$

$$\frac{\partial E}{\partial b^{(\ell)}} = \delta^{(\ell)}$$

6.4 Properties of Multi-Layer Perceptrons

Notes

Multi-Layer Perceptrons haben gegenüber einzelnen Perzeptrons deutlich stärkere Ausdrucksfähigkeit.

Die wichtigsten Eigenschaften sind:

- mehrere Schichten,
- nichtlineare Aktivierungsfunktionen,
- lernbare Gewichtsmatrizen und Biases,

- hierarchische Repräsentationen,
- flexible Entscheidungsgrenzen.

Solution

Ein einzelnes Perzeptron kann nur lineare Entscheidungsgrenzen darstellen:

$$w^T x + b = 0.$$

Ein MLP kann dagegen durch versteckte Schichten eine Eingabe zuerst in neue Merkmalsräume transformieren.

Dadurch kann ein ursprünglich nicht linear separierbares Problem in einer versteckten Repräsentation linear separierbar werden.

Answer

Die zentrale Eigenschaft von MLPs ist:

MLPs lernen nicht nur eine Entscheidung, sondern auch eine geeignete Repräsentation der Eingabe.

Dadurch können sie komplexere Funktionen modellieren als einfache lineare Klassifikatoren.

6.5 MLPs and the XOR Problem

Notes

Das XOR-Problem ist das klassische Beispiel dafür, warum mehrere Schichten und Nichtlinearitäten nützlich sind.

Die XOR-Funktion ist:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1,$$

$$1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

Die beiden positiven Punkte liegen diagonal gegenüber. Daher ist XOR nicht linear separierbar.

Solution

Ein einzelnes Perzeptron kann XOR nicht lösen, weil es nur eine Gerade als Entscheidungsgrenze erzeugen kann.

Ein MLP mit versteckter Schicht kann dagegen mehrere lineare Schnitte kombinieren und danach nichtlinear transformieren.

Dadurch kann es eine nichtlineare Entscheidungsregion erzeugen, die XOR korrekt klassifiziert.

Answer

XOR zeigt den Unterschied zwischen Perzeptron und MLP:

Perzeptron: nur linear separierbare Probleme

MLP: nichtlineare Entscheidungsgrenzen durch versteckte Schichten

6.6 Overfitting and Generalization Error for MLPs

Notes

MLPs sind sehr flexible Modelle. Diese Flexibilität ist einerseits nützlich, kann aber auch zu Overfitting führen.

Overfitting bedeutet:

Das Modell passt die Trainingsdaten sehr gut an, generalisiert aber schlecht.

Typisches Muster:

$$E_{\text{train}} \text{ klein, } E_{\text{test}} \text{ groß.}$$

Solution

Ein MLP kann viele Parameter haben. Wenn das Modell im Vergleich zur Datenmenge zu komplex ist, kann es nicht nur die zugrundeliegende Struktur, sondern auch das Rauschen der Trainingsdaten lernen.

Dann sinkt der Trainingsfehler weiter, während der Fehler auf neuen Daten steigt.

Das Ziel ist daher nicht nur, den Trainingsfehler zu minimieren, sondern eine gute Generalisierung zu erreichen.

Calculation

Der Generalization Error beschreibt den erwarteten Fehler auf neuen, unabhängigen Daten:

$$E_{\text{gen}} = \mathbb{E}_{(x,t) \sim p_{\text{data}}} [L(f_{\theta}(x), t)].$$

Dieser Fehler ist in der Praxis unbekannt, da die wahre Datenverteilung p_{data} unbekannt ist.

Deshalb approximiert man ihn durch einen Test- oder Validierungsfehler:

$$E_{\text{val}} = \frac{1}{N_{\text{val}}} \sum_{n=1}^{N_{\text{val}}} L(f_{\theta}(x_n), t_n).$$

Answer

Overfitting erkennt man typischerweise daran:

$$E_{\text{train}} \downarrow \quad \text{aber} \quad E_{\text{val}} \uparrow$$

Gute Modelle sollen nicht nur auf Trainingsdaten gut sein, sondern auf neuen Daten:

Ziel: kleiner Generalization Error.

6.7 Validation Sets for MLPs

Notes

Ein Validation Set ist ein Teil der Daten, der nicht zum direkten Training der Gewichte verwendet wird.

Es dient dazu, Modellentscheidungen zu treffen, zum Beispiel:

- Wahl der Netzwerkgröße,
- Wahl der Lernrate,
- Wahl der Regularisierung,
- Entscheidung über Early Stopping,
- Vergleich verschiedener Modelle.

Solution

Typische Aufteilung der Daten:

Training Set Validation Set Test Set.

Das Training Set wird verwendet, um Parameter wie Gewichte und Biases zu lernen.

Das Validation Set wird verwendet, um Hyperparameter und Modellvarianten zu wählen.

Das Test Set wird erst am Ende verwendet, um die finale Generalisierung zu bewerten.

Answer

Die Rollen der Datensätze sind:

Training Set: Parameter lernen

Validation Set: Hyperparameter und Modellwahl

Test Set: finale Bewertung

Das Test Set sollte nicht zur Modellwahl verwendet werden.

6.8 Part C.2: MLPs and Maximum Likelihood Estimation

Notes

MLPs können probabilistisch interpretiert werden.

Dazu modelliert das Netzwerk Parameter einer Wahrscheinlichkeitsverteilung.

Für Klassifikation gibt ein MLP zum Beispiel Klassenwahrscheinlichkeiten aus:

$$p(t = k \mid x, \theta).$$

Für Regression kann ein MLP den Mittelwert einer Gauß-Verteilung modellieren:

$$p(t \mid x, \theta) = \mathcal{N}(t \mid f_{\theta}(x), \sigma^2).$$

Calculation

Für gelabelte Daten

$$\mathcal{D} = \{(x_1, t_1), \dots, (x_N, t_N)\}$$

lautet die Likelihood:

$$L(\theta) = \prod_{n=1}^N p(t_n \mid x_n, \theta).$$

Die Log-Likelihood ist:

$$\ell(\theta) = \sum_{n=1}^N \log p(t_n \mid x_n, \theta).$$

Maximum Likelihood bedeutet:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \ell(\theta).$$

Calculation

In der Praxis minimiert man oft die negative Log-Likelihood:

$$E(\theta) = - \sum_{n=1}^N \log p(t_n \mid x_n, \theta).$$

Für binäre Klassifikation mit

$$y_n = p(t_n = 1 \mid x_n, \theta)$$

erhält man die Cross-Entropy:

$$E(\theta) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Answer

MLP-Training kann als Maximum Likelihood formuliert werden:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(t_n | x_n, \theta)$$

Äquivalent:

$$\hat{\theta}_{\text{ML}} = \arg \min_{\theta} \left[- \sum_{n=1}^N \log p(t_n | x_n, \theta) \right]$$

Viele Loss-Funktionen entstehen also aus probabilistischen Annahmen.

6.9 Part C.3: Generalization and Validation

Notes

Generalisierung beschreibt die Fähigkeit eines Modells, auf neuen, ungesehenen Daten gut zu funktionieren.

Ein Modell soll also nicht nur die Trainingsdaten auswendig lernen, sondern die zugrundeliegende Struktur der Daten erfassen.

Solution

Generalisierung hängt von mehreren Faktoren ab:

- Größe und Qualität des Trainingsdatensatzes,
- Modellkomplexität,
- Regularisierung,
- Optimierungsverfahren,
- Wahl der Hyperparameter,
- Verteilung von Trainings- und Testdaten.

Ein Validation Set hilft dabei, diese Faktoren praktisch zu kontrollieren.

Calculation

Häufig betrachtet man drei Fehlergrößen:

$$E_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} L(f_{\theta}(x_n), t_n),$$

$$E_{\text{val}} = \frac{1}{N_{\text{val}}} \sum_{n=1}^{N_{\text{val}}} L(f_{\theta}(x_n), t_n),$$

$$E_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{n=1}^{N_{\text{test}}} L(f_{\theta}(x_n), t_n).$$

Der Trainingsfehler misst Anpassung an die Trainingsdaten.

Der Validierungsfehler hilft bei Modellwahl.

Der Testfehler schätzt die finale Generalisierung.

Answer

Gute Modellwahl bedeutet:

Nicht das Modell mit minimalem Trainingsfehler wählen, sondern das mit gutem Validierungsfehler.

Ein Validation Set dient zur Modell- und Hyperparameterwahl.

Ein Test Set dient zur finalen unabhängigen Bewertung.

6.10 Regularization and Early Stopping

Notes

Regularisierung bezeichnet Methoden, die Overfitting reduzieren sollen.

Sie schränken das Modell direkt oder indirekt ein, damit es besser generalisiert.

Solution

Typische Regularisierungsmethoden für MLPs sind:

- Gewichtsnorm-Strafen wie L^2 -Regularisierung,
- kleinere Netzwerke,
- Dropout,
- Data Augmentation,
- Early Stopping.

Bei Early Stopping beobachtet man den Validierungsfehler während des Trainings.

Wenn der Trainingsfehler weiter sinkt, aber der Validierungsfehler wieder steigt, stoppt man das Training.

Calculation

Ein typischer regularisierter Loss hat die Form:

$$E_{\text{reg}}(\theta) = E_{\text{data}}(\theta) + \lambda\Omega(\theta).$$

Für L^2 -Regularisierung:

$$\Omega(\theta) = \frac{1}{2}\|\theta\|^2.$$

Also:

$$E_{\text{reg}}(\theta) = E_{\text{data}}(\theta) + \frac{\lambda}{2}\|\theta\|^2.$$

Answer

Regularisierung soll Overfitting reduzieren:

$$E_{\text{reg}}(\theta) = E_{\text{data}}(\theta) + \lambda\Omega(\theta)$$

Early Stopping nutzt den Validierungsfehler, um das Training rechtzeitig zu beenden.

6.11 Part C.4: Other Neural Networks

Notes

MLPs sind nicht die einzige Art neuronaler Netze.

Für verschiedene Datenarten und Aufgaben verwendet man spezialisierte Architekturen.

Solution

Beispiele für andere neuronale Netzwerke sind:

- **Convolutional Neural Networks:** besonders wichtig für Bilder und räumliche Daten.
- **Recurrent Neural Networks:** historisch wichtig für Sequenzen und Zeitreihen.
- **Long Short-Term Memory Networks:** spezielle rekurrente Netze zur Verarbeitung längerer Abhängigkeiten.
- **Autoencoder:** Netze, die Daten komprimieren und rekonstruieren lernen.
- **Transformer:** Architekturen mit Attention-Mechanismus, besonders wichtig für Sprache und viele moderne Deep-Learning-Systeme.

Answer

Unterschiedliche neuronale Netzwerkarchitekturen sind für unterschiedliche Datenstrukturen geeignet:

MLPs: allgemeine Vektordaten

CNNs: Bilder und räumliche Muster

RNNs/LSTMs/Transformer: Sequenzen

Autoencoder: Repräsentationslernen und Rekonstruktion

6.12 Prüfungsrelevante Kurzfassung

Answer

Lecture 6 behandelt DNNs und ihre Fähigkeiten.

Die wichtigsten Punkte sind:

- Ein MLP besteht aus Schichten:

$$a^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = g^{(\ell)}(a^{(\ell)}).$$

- Backpropagation berechnet Gradienten effizient mit der Kettenregel.
- Der Fehlerterm einer Schicht ist:

$$\delta^{(\ell)} = \frac{\partial E}{\partial a^{(\ell)}}.$$

- Für versteckte Schichten gilt:

$$\delta^{(\ell)} = \left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right) \odot g'^{(\ell)}(a^{(\ell)}).$$

- Gewichtgradient:

$$\frac{\partial E}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(h^{(\ell-1)} \right)^T.$$

- MLPs können XOR lösen, weil sie nichtlineare versteckte Repräsentationen lernen.
- Overfitting bedeutet:

$$E_{\text{train}} \text{ klein,} \quad E_{\text{val/test}} \text{ groß.}$$

- Validation Sets dienen zur Modellwahl und Hyperparameterwahl.
- MLP-Training kann als Maximum Likelihood formuliert werden:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{n=1}^N \log p(t_n | x_n, \theta).$$

- Negative Log-Likelihood wird als Loss minimiert:

$$E(\theta) = - \sum_{n=1}^N \log p(t_n | x_n, \theta).$$

- Regularisierung reduziert Overfitting:

$$E_{\text{reg}}(\theta) = E_{\text{data}}(\theta) + \lambda \Omega(\theta).$$

- Andere Netztypen sind zum Beispiel CNNs, RNNs, LSTMs, Autoencoder und Transformer.

Die Kernaussage lautet:

DNNs sind leistungsfähig, weil sie durch viele Schichten hierarchische nichtlineare Repräsentationen lernen

7 Lecture 7: Dimensionality Reduction with PCA

7.1 Overview

Notes

Lecture 7 behandelt Dimensionsreduktion mit Principal Component Analysis, kurz PCA. Die zentrale Idee ist:

Hochdimensionale Daten sollen durch wenige informative Richtungen beschrieben werden.

Die wichtigsten Themen sind:

- Projektionsoperatoren,
- das eindimensionale PCA-Problem,
- Maximum-Variance-Ansatz,
- Herleitung der Varianzformel,
- optimale Lösung über Eigenvektoren,
- das allgemeine niedrigdimensionale Hyperplane-Problem,
- Anwendungen von PCA,
- Diskussion der Stärken und Grenzen von PCA.

Answer

Die Kernaussage von Lecture 7 ist:

PCA findet Richtungen maximaler Varianz und projiziert Daten auf diese Richtungen.

Dadurch kann man hochdimensionale Daten visualisieren, interpretieren oder komprimieren.

7.2 Preparation: Projection Operators

Notes

Eine Projektion bildet einen Vektor auf einen Unterraum ab.

Für PCA interessiert uns besonders die Projektion eines Datenpunkts x auf eine Richtung u .

Dabei nehmen wir an, dass u normiert ist:

$$\|u\| = 1.$$

Calculation

Die Projektion eines Vektors $x \in \mathbb{R}^D$ auf die Richtung u ist:

$$\text{proj}_u(x) = (u^T x)u.$$

Der Skalar

$$z = u^T x$$

ist die Koordinate von x entlang der Richtung u .

Die Matrixform des Projektionsoperators ist:

$$P_u = uu^T.$$

Damit:

$$\text{proj}_u(x) = P_u x = uu^T x.$$

Solution

Wenn die Daten nicht um den Ursprung zentriert sind, projiziert man normalerweise die zentrierten Daten.

Für Datenpunkte

$$x_1, \dots, x_N$$

berechnet man zuerst den Mittelwert:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Danach betrachtet man:

$$\tilde{x}_n = x_n - \bar{x}.$$

PCA wird typischerweise auf diesen zentrierten Daten durchgeführt.

Answer

Projektion auf eine normierte Richtung u :

$$\text{proj}_u(x) = (u^T x)u$$

Projektionsmatrix:

$$P_u = uu^T$$

Zentrierte Daten:

$$\tilde{x}_n = x_n - \bar{x}$$

7.3 The One-Dimensional Problem: Finding a Line Through Data

Notes

Im eindimensionalen PCA-Problem sucht man eine Gerade, auf die die Daten projiziert werden sollen.

Die Gerade soll möglichst viel Information der Daten erhalten.

PCA formalisiert diese Idee durch den Maximum-Variance-Ansatz:

Wähle die Richtung, in der die projizierten Daten maximale Varianz haben.

Solution

Gegeben seien zentrierte Datenpunkte:

$$\tilde{x}_1, \dots, \tilde{x}_N \in \mathbb{R}^D.$$

Wir suchen eine normierte Richtung

$$u \in \mathbb{R}^D, \quad \|u\| = 1.$$

Jeder Datenpunkt wird auf diese Richtung projiziert. Die eindimensionale Koordinate des

Datenpunkts entlang u ist:

$$z_n = u^T \tilde{x}_n.$$

PCA sucht die Richtung u , sodass die Varianz der z_n maximal wird.

Answer

Das eindimensionale PCA-Problem lautet:

Finde eine Richtung u mit $\|u\| = 1$, sodass die Projektionen $u^T \tilde{x}_n$ maximale Varianz haben.

7.4 Maximum Variance Approach in One Dimension

Notes

Die Varianz der projizierten Daten misst, wie stark sich die Daten entlang der Richtung u ausbreiten.

Eine Richtung mit großer Varianz enthält viel Struktur der Daten.

Eine Richtung mit kleiner Varianz enthält wenig Variation und ist für die Beschreibung der Daten weniger informativ.

Calculation

Die Projektionskoordinate des n -ten Datenpunkts ist:

$$z_n = u^T \tilde{x}_n.$$

Da die Daten zentriert sind, gilt:

$$\frac{1}{N} \sum_{n=1}^N \tilde{x}_n = 0.$$

Daher ist auch der Mittelwert der Projektionen null:

$$\bar{z} = \frac{1}{N} \sum_{n=1}^N z_n = \frac{1}{N} \sum_{n=1}^N u^T \tilde{x}_n = u^T \left(\frac{1}{N} \sum_{n=1}^N \tilde{x}_n \right) = 0.$$

Calculation

Die Varianz der Projektionen ist daher:

$$\text{Var}(z) = \frac{1}{N} \sum_{n=1}^N z_n^2.$$

Einsetzen von

$$z_n = u^T \tilde{x}_n$$

liefert:

$$\text{Var}(z) = \frac{1}{N} \sum_{n=1}^N (u^T \tilde{x}_n)^2.$$

Da

$$(u^T \tilde{x}_n)^2 = u^T \tilde{x}_n \tilde{x}_n^T u,$$

folgt:

$$\text{Var}(z) = u^T \left(\frac{1}{N} \sum_{n=1}^N \tilde{x}_n \tilde{x}_n^T \right) u.$$

Calculation

Die Matrix

$$S = \frac{1}{N} \sum_{n=1}^N \tilde{x}_n \tilde{x}_n^T$$

ist die empirische Kovarianzmatrix der Daten.

Damit lautet die Varianz entlang der Richtung u :

$$\text{Var}(z) = u^T S u.$$

Das Optimierungsproblem ist:

$$\max_u u^T S u \quad \text{unter der Nebenbedingung} \quad u^T u = 1.$$

Answer

Die Varianz der Projektionen auf u ist:

$$\boxed{\text{Var}(z) = u^T S u}$$

mit Kovarianzmatrix:

$$\boxed{S = \frac{1}{N} \sum_{n=1}^N \tilde{x}_n \tilde{x}_n^T}$$

Das PCA-Optimierungsproblem ist:

$$\boxed{\max_u u^T S u \quad \text{s.t.} \quad u^T u = 1.}$$

7.5 Optimal Solution in Terms of Eigenvectors

Notes

Das PCA-Optimierungsproblem führt auf ein Eigenwertproblem.

Die optimale Richtung ist der Eigenvektor der Kovarianzmatrix zum größten Eigenwert.

Calculation

Wir maximieren:

$$u^T S u$$

unter der Nebenbedingung:

$$u^T u = 1.$$

Dafür verwenden wir einen Lagrange-Multiplikator λ :

$$\mathcal{L}(u, \lambda) = u^T S u - \lambda(u^T u - 1).$$

Ableitung nach u :

$$\frac{\partial \mathcal{L}}{\partial u} = 2S u - 2\lambda u.$$

Setze gleich null:

$$2S u - 2\lambda u = 0.$$

Also:

$$S u = \lambda u.$$

Solution

Die Gleichung

$$S u = \lambda u$$

ist die Eigenwertgleichung der Kovarianzmatrix S .

Für einen normierten Eigenvektor u gilt:

$$u^T S u = u^T \lambda u = \lambda u^T u = \lambda.$$

Die Varianz entlang einer Eigenrichtung ist also genau der zugehörige Eigenwert.

Daher maximiert der Eigenvektor mit dem größten Eigenwert die Varianz.

Answer

Die erste Hauptkomponente ist:

$$u_1 = \text{Eigenvektor von } S \text{ zum größten Eigenwert } \lambda_1.$$

Die erklärte Varianz entlang dieser Richtung ist:

$$\text{Var}(u_1^T \tilde{x}) = \lambda_1.$$

7.6 The General Problem: Finding a Low-Dimensional Hyperplane

Notes

Im allgemeinen PCA-Problem sucht man nicht nur eine Richtung, sondern einen niedrig-dimensionalen Unterraum.

Ziel:

Finde einen M -dimensionalen Unterraum in $\mathbb{R}^D$, $M < D$,

der möglichst viel Varianz der Daten erhält.

Solution

Statt einer Richtung u betrachtet man mehrere orthonormale Richtungen:

$$u_1, \dots, u_M.$$

Diese bilden die Spalten einer Matrix:

$$U_M = \begin{pmatrix} | & & | \\ u_1 & \cdots & u_M \\ | & & | \end{pmatrix} \in \mathbb{R}^{D \times M}.$$

Die Orthonormalität bedeutet:

$$U_M^T U_M = I_M.$$

Die Projektion eines zentrierten Datenpunkts $\tilde{x}$ auf den M -dimensionalen Unterraum ist:

$$z = U_M^T \tilde{x}.$$

Dabei gilt:

$$z \in \mathbb{R}^M.$$

Answer

Allgemeine PCA sucht eine Matrix

$$U_M = (u_1, \dots, u_M)$$

mit

$$U_M^T U_M = I_M$$

sodass die Projektionen

$$z_n = U_M^T \tilde{x}_n$$

möglichst viel Varianz erhalten.

7.7 Maximum Variance Approach in the General Case

Calculation

Die Gesamtvarianz der Projektionen auf den Unterraum ist die Summe der Varianzen entlang der gewählten Richtungen:

$$J(U_M) = \sum_{j=1}^M u_j^T S u_j.$$

In Matrixform kann man das schreiben als:

$$J(U_M) = \text{tr}(U_M^T S U_M).$$

Das Optimierungsproblem lautet:

$$\max_{U_M} \text{tr}(U_M^T S U_M)$$

unter der Nebenbedingung:

$$U_M^T U_M = I_M.$$

Solution

Die optimale Lösung wird durch die Eigenvektoren der Kovarianzmatrix gegeben.
Sei:

$$S u_j = \lambda_j u_j$$

mit geordneten Eigenwerten:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D.$$

Dann wählt PCA die ersten M Eigenvektoren:

$$u_1, \dots, u_M.$$

Diese Richtungen enthalten maximal viel Varianz unter allen M -dimensionalen linearen Projektionen.

Answer

Die allgemeine PCA-Lösung ist:

$$U_M = (u_1, \dots, u_M)$$

wobei $u_1, \dots, u_M$ die Eigenvektoren von S zu den größten Eigenwerten sind:

$$Su_j = \lambda_j u_j, \quad \lambda_1 \geq \lambda_2 \geq \dots$$

Die erhaltene Varianz ist:

$$\sum_{j=1}^M \lambda_j.$$

7.8 Projection and Reconstruction

Notes

PCA kann auch als Projektion und anschließende Rekonstruktion verstanden werden. Man reduziert zuerst die Dimension und rekonstruiert danach näherungsweise die ursprünglichen Daten.

Calculation

Für einen zentrierten Datenpunkt $\tilde{x}$ ist die niedrigdimensionale Darstellung:

$$z = U_M^T \tilde{x}.$$

Die Rekonstruktion im ursprünglichen Raum ist:

$$\hat{x} = U_M z + \bar{x}.$$

Einsetzen von z :

$$\hat{x} = U_M U_M^T \tilde{x} + \bar{x}.$$

Für den ursprünglichen Datenpunkt x gilt:

$$\tilde{x} = x - \bar{x}.$$

Also:

$$\hat{x} = U_M U_M^T (x - \bar{x}) + \bar{x}.$$

Answer

PCA-Projektion:

$$z = U_M^T (x - \bar{x})$$

PCA-Rekonstruktion:

$$\hat{x} = U_M z + \bar{x} = U_M U_M^T (x - \bar{x}) + \bar{x}$$

7.9 Explained Variance

Notes

Die Eigenwerte der Kovarianzmatrix geben an, wie viel Varianz entlang der jeweiligen Hauptkomponenten liegt.

Große Eigenwerte bedeuten wichtige Richtungen.

Kleine Eigenwerte bedeuten Richtungen mit wenig Variation.

Calculation

Die gesamte Varianz der Daten ist:

$$\sum_{j=1}^D \lambda_j.$$

Wenn man nur die ersten M Hauptkomponenten verwendet, ist die erklärte Varianz:

$$\sum_{j=1}^M \lambda_j.$$

Der Anteil erklärter Varianz ist daher:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}.$$

Answer

Anteil erklärter Varianz:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}$$

Diese Größe hilft bei der Wahl der reduzierten Dimension M .

7.10 Example Applications

Notes

PCA wird in vielen Bereichen verwendet, wenn hochdimensionale Daten analysiert oder vereinfacht werden sollen.

Typische Anwendungen sind:

- niedrigdimensionale Visualisierung hochdimensionaler Daten,
- Dateninterpretation,
- Datenkompression.

Solution

Visualisierung:

Hochdimensionale Daten können auf zwei oder drei Hauptkomponenten projiziert werden. Dadurch kann man sie in 2D oder 3D darstellen:

$$x_n \in \mathbb{R}^D \longrightarrow z_n \in \mathbb{R}^2 \text{ oder } \mathbb{R}^3.$$

So kann man Cluster, Ausreißer oder grobe Strukturen erkennen.

Solution

Dateninterpretation:

Die Hauptkomponenten zeigen Richtungen, in denen die Daten besonders stark variieren. Wenn die ursprünglichen Features interpretierbar sind, kann man untersuchen, welche Features stark zu einer Hauptkomponente beitragen.

Dadurch können wichtige Variationsmuster sichtbar werden.

Solution

Datenkompression:

Wenn die ersten wenigen Hauptkomponenten bereits einen großen Teil der Varianz erklären, kann man die Daten in reduzierter Dimension speichern.

Statt eines D -dimensionalen Vektors speichert man nur:

$$z \in \mathbb{R}^M, \quad M \ll D.$$

Die Rekonstruktion ist dann näherungsweise möglich durch:

$$\hat{x} = U_M z + \bar{x}.$$

Answer

PCA-Anwendungen:

- **Visualisierung:** Projektion auf zwei oder drei Dimensionen.
- **Interpretation:** Analyse wichtiger Varianzrichtungen.
- **Kompression:** Speicherung der Daten mit weniger Koordinaten.

PCA ersetzt hochdimensionale Daten durch:

$$z = U_M^T(x - \bar{x}) \in \mathbb{R}^M, \quad M \ll D.$$

7.11 Discussion of PCA

Notes

PCA ist ein lineares Verfahren zur Dimensionsreduktion.

Es ist einfach, effizient und gut interpretierbar, hat aber auch Grenzen.

Solution

Vorteile von PCA:

- rechnerisch gut handhabbar,
- klare geometrische Interpretation,
- keine Labels nötig,
- nützlich für Visualisierung und Kompression,
- optimale lineare Projektion im Sinne maximaler Varianz.

Solution

Grenzen von PCA:

- PCA ist linear und findet keine nichtlinearen Strukturen.
- Große Varianz muss nicht immer für die Aufgabe relevant sein.
- PCA ist empfindlich gegenüber Skalierung der Features.
- PCA ist empfindlich gegenüber Ausreißern.
- Die Hauptkomponenten können schwer interpretierbar sein.

Answer

PCA ist besonders sinnvoll, wenn:

wichtige Struktur der Daten in linearen Varianzrichtungen liegt.

PCA ist weniger geeignet, wenn:

die relevante Struktur stark nichtlinear ist oder nicht mit maximaler Varianz zusammenhängt.

7.12 Prüfungsrelevante Kurzfassung

Answer

Lecture 7 behandelt PCA als lineare Dimensionsreduktion.

Die wichtigsten Punkte sind:

- Daten werden zuerst zentriert:

$$\tilde{x}_n = x_n - \bar{x}.$$

- Die Kovarianzmatrix ist:

$$S = \frac{1}{N} \sum_{n=1}^N \tilde{x}_n \tilde{x}_n^T.$$

- Projektion auf eine normierte Richtung u :

$$z_n = u^T \tilde{x}_n.$$

- Die Varianz der Projektionen ist:

$$\text{Var}(z) = u^T S u.$$

- Das eindimensionale PCA-Problem ist:

$$\max_u u^T S u \quad \text{s.t.} \quad u^T u = 1.$$

- Die Lösung erfüllt:

$$S u = \lambda u.$$

- Die erste Hauptkomponente ist der Eigenvektor zum größten Eigenwert.
- Für M -dimensionale PCA wählt man die ersten M Eigenvektoren:

$$U_M = (u_1, \dots, u_M).$$

- Projektion:

$$z = U_M^T(x - \bar{x}).$$

- Rekonstruktion:

$$\hat{x} = U_M U_M^T(x - \bar{x}) + \bar{x}.$$

- Anteil erklärter Varianz:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}.$$

- Anwendungen: Visualisierung, Interpretation und Kompression.

Die Kernaussage lautet:

PCA projiziert Daten auf die Eigenrichtungen der Kovarianzmatrix mit größter Varianz.

8 Lecture 8: Clustering – k-means Algorithms, Hierarchical Clustering and GMMs

8.1 Overview

Notes

Lecture 8 behandelt Clustering-Verfahren, also Methoden zum Finden von Gruppen in ungelabelten Daten.

Die zentralen Themen sind:

- das Problem, Cluster in Daten zu finden,
- k-means,
- ein ad-hoc Objective für k-means,
- Lloyd's Algorithmus,
- Fuzzy-k-means,
- hierarchisches Clustering,
- agglomeratives Clustering,
- Gaussian Mixture Models,
- Maximum-Likelihood-Schätzung für GMMs,
- Free Energy beziehungsweise ELBO als Hilfsobjective,
- Update des Mittelwerts in GMMs,
- Diskussion der Methoden.

Answer

Die Kernaussage von Lecture 8 ist:

Clustering versucht, ohne Labels sinnvolle Gruppenstrukturen in Daten zu finden.

k-means beschreibt Cluster durch harte Zuordnungen zu Zentren, während GMMs Cluster probabilistisch als Mischung von Verteilungen modellieren.

8.2 The Problem of Finding Clusters in Data

Notes

Beim Clustering sind keine Labels gegeben.

Man beobachtet nur Datenpunkte:

$$x_1, \dots, x_N \in \mathbb{R}^D.$$

Ziel ist es, diese Datenpunkte in Gruppen einzuteilen, sodass Punkte innerhalb derselben Gruppe einander ähnlich sind und Punkte aus verschiedenen Gruppen möglichst unterschiedlich sind.

Solution

Clustering ist ein Beispiel für *unsupervised learning*.

Im Gegensatz zur Klassifikation gibt es keine bekannten Zielwerte:

$$(x_n, t_n)$$

sondern nur:

$$x_n.$$

Das Modell muss also selbst aus der Struktur der Daten ableiten, welche Gruppen sinnvoll sind.

Typische Anwendungen sind:

- Gruppierung von Kundendaten,
- Segmentierung von Bildern,
- Finden biologischer Zelltypen,
- Strukturierung großer Datensätze,
- Vorverarbeitung für weitere Machine-Learning-Methoden.

Answer

Clustering bedeutet:

Finde Gruppen in ungelabelten Daten $x_1, \dots, x_N$.

Die Schwierigkeit ist, dass nicht vorgegeben ist, was ein “guter” Cluster genau ist. Daher braucht man ein Objective oder ein probabilistisches Modell.

8.3 k-means: Basic Idea

Notes

k-means ist eines der einfachsten und wichtigsten Clustering-Verfahren.

Man nimmt an, dass es K Cluster gibt.

Jeder Cluster wird durch ein Zentrum beschrieben:

$$\mu_1, \dots, \mu_K \in \mathbb{R}^D.$$

Jeder Datenpunkt wird genau einem Clusterzentrum zugeordnet.

Solution

Die Grundidee von k-means ist:

- Wähle K Clusterzentren.

- Ordne jeden Datenpunkt dem nächstgelegenen Zentrum zu.
- Aktualisiere jedes Zentrum als Mittelwert der ihm zugeordneten Datenpunkte.
- Wiederhole diese Schritte bis zur Konvergenz.

k-means erzeugt also eine harte Clusterzuordnung:

$$x_n \mapsto k_n, \quad k_n \in \{1, \dots, K\}.$$

Answer

Bei k-means wird jeder Datenpunkt genau einem Cluster zugeordnet:

$$k_n = \arg \min_k \|x_n - \mu_k\|^2$$

Danach wird jedes Zentrum als Mittelwert seiner zugeordneten Punkte aktualisiert.

8.4 k-means: Definition of an Ad-hoc Objective

Notes

k-means kann über ein ad-hoc Objective formuliert werden.

Die Idee ist:

Punkte sollen möglichst nahe an ihren jeweiligen Clusterzentren liegen.

Dafür minimiert man die Summe der quadratischen Abstände zu den Clusterzentren.

Calculation

Sei r_{nk} eine binäre Zuordnungsvariable:

$$r_{nk} = \begin{cases} 1, & \text{wenn } x_n \text{ Cluster } k \text{ zugeordnet ist,} \\ 0, & \text{sonst.} \end{cases}$$

Für jeden Datenpunkt gilt:

$$\sum_{k=1}^K r_{nk} = 1.$$

Das k-means Objective lautet:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2.$$

Ziel ist:

$$\min_{\{r_{nk}\}, \{\mu_k\}} J.$$

Answer

Das k-means Objective ist:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2$$

Dabei ist r_{nk} die harte Clusterzuordnung und μ_k das Zentrum von Cluster k .

8.5 Derivation of Lloyd's Algorithm

Notes

Lloyd's Algorithmus ist der Standardalgorithmus für k-means.

Er minimiert das k-means Objective iterativ durch abwechselnde Optimierung:

- Fixiere die Zentren und optimiere die Zuordnungen.
- Fixiere die Zuordnungen und optimiere die Zentren.

Calculation

Schritt 1: Update der Zuordnungen.

Sind die Zentren $\mu_1, \dots, \mu_K$ fixiert, minimiert man für jeden Datenpunkt:

$$\sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2.$$

Da jeder Punkt genau einem Cluster zugeordnet wird, wählt man den Cluster mit dem kleinsten Abstand:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|x_n - \mu_j\|^2, \\ 0, & \text{sonst.} \end{cases}$$

Calculation

Schritt 2: Update der Zentren.

Sind die Zuordnungen r_{nk} fixiert, minimiert man

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2$$

bezüglich μ_k .

Für ein fixes k betrachtet man:

$$J_k = \sum_{n=1}^N r_{nk} \|x_n - \mu_k\|^2.$$

Ableitung nach μ_k :

$$\frac{\partial J_k}{\partial \mu_k} = -2 \sum_{n=1}^N r_{nk} (x_n - \mu_k).$$

Gleich null setzen:

$$\sum_{n=1}^N r_{nk} (x_n - \mu_k) = 0.$$

Also:

$$\sum_{n=1}^N r_{nk} x_n = \mu_k \sum_{n=1}^N r_{nk}.$$

Damit:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}.$$

Answer

Lloyd's Algorithmus besteht aus zwei Updates:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|x_n - \mu_j\|^2, \\ 0, & \text{sonst} \end{cases}$$

und

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}$$

Der Algorithmus verringert das k-means Objective in jedem Schritt oder lässt es unverändert.

8.6 Lloyd's Algorithm: Practical Procedure

Solution

Praktisches Vorgehen:

1. Wähle K , die Anzahl der Cluster.
2. Initialisiere die Zentren $\mu_1, \dots, \mu_K$, zum Beispiel zufällig.
3. Ordne jeden Datenpunkt dem nächsten Zentrum zu.

4. Aktualisiere jedes Zentrum als Mittelwert seiner zugeordneten Punkte.
5. Wiederhole Zuordnung und Update bis sich nichts mehr ändert oder bis ein Abbruchkriterium erreicht ist.

Notes

k-means findet nicht garantiert das globale Minimum des Objectives.
Das Ergebnis hängt von der Initialisierung ab.
Deshalb startet man k-means in der Praxis oft mehrfach mit verschiedenen Initialisierungen und wählt die Lösung mit dem kleinsten Objective.

Answer

k-means ist einfach und effizient, aber:

Das Ergebnis hängt von K und der Initialisierung ab.

Außerdem bevorzugt k-means ungefähr kugelförmige Cluster ähnlicher Größe.

8.7 Fuzzy-k-means

Notes

Beim normalen k-means gehört jeder Datenpunkt genau zu einem Cluster.
Das nennt man harte Zuordnung.
Fuzzy-k-means erlaubt dagegen weiche Zuordnungen:

$$r_{nk} \in [0,1].$$

Dabei beschreibt r_{nk} , wie stark der Datenpunkt x_n zu Cluster k gehört.

Solution

Für jeden Datenpunkt gilt weiterhin:

$$\sum_{k=1}^K r_{nk} = 1.$$

Aber jetzt darf ein Punkt teilweise mehreren Clustern angehören.
Das ist nützlich, wenn Cluster überlappen oder wenn Zuordnungen unsicher sind.
Fuzzy-k-means ist damit ein Zwischenschritt zwischen hartem k-means und probabilistischen Mischmodellen wie Gaussian Mixture Models.

Calculation

Ein typisches Fuzzy-k-means Objective ist:

$$J_m = \sum_{n=1}^N \sum_{k=1}^K r_{nk}^m \|x_n - \mu_k\|^2.$$

Dabei ist

$$m > 1$$

ein Fuzziness-Parameter.

Für größere m werden die Zuordnungen weicher.

Answer

Fuzzy-k-means ersetzt harte Zuordnungen durch weiche Memberships:

$$r_{nk} \in [0,1], \quad \sum_{k=1}^K r_{nk} = 1.$$

Dadurch kann ein Punkt teilweise zu mehreren Clustern gehören.

8.8 Hierarchical Clustering

Notes

Hierarchisches Clustering erzeugt nicht nur eine einzelne Clusterlösung, sondern eine Hierarchie von Clustern.

Das Ergebnis wird oft als Dendrogramm dargestellt.

Es gibt zwei Haupttypen:

- agglomeratives Clustering,
- divisives Clustering.

Solution

Beim **agglomerativen Clustering** startet man mit vielen kleinen Clustern.

Meist startet jeder Datenpunkt als eigener Cluster.

Dann werden schrittweise die ähnlichsten Cluster zusammengeführt.

Beim **divisiven Clustering** startet man mit einem großen Cluster und teilt diesen schrittweise auf.

In der Praxis ist agglomeratives Clustering besonders verbreitet.

Answer

Hierarchisches Clustering liefert eine Clusterhierarchie:

$$\text{kleine Cluster} \longleftrightarrow \text{große Cluster}$$

Ein Dendrogramm zeigt, bei welchen Distanzen Cluster zusammengeführt werden.

8.9 Agglomerative Clustering

Notes

Agglomeratives Clustering ist ein bottom-up Verfahren.

Es startet mit N Clustern:

$$C_1 = \{x_1\}, \dots, C_N = \{x_N\}.$$

Danach werden wiederholt die zwei ähnlichsten Cluster zusammengeführt.

Solution

Algorithmus:

1. Starte mit jedem Datenpunkt als eigenem Cluster.
2. Berechne Distanzen zwischen allen Clustern.
3. Führe die zwei nächstgelegenen Cluster zusammen.
4. Aktualisiere die Distanzen.
5. Wiederhole, bis nur noch ein Cluster übrig ist oder bis die gewünschte Anzahl von Clustern erreicht ist.

Notes

Entscheidend ist, wie man die Distanz zwischen zwei Clustern definiert.

Häufige Linkage-Kriterien sind:

- single linkage,
- complete linkage,
- average linkage,
- centroid linkage,
- Ward linkage.

Calculation

Beispiele für Cluster-Distanzen:

Single linkage:

$$d(C_a, C_b) = \min_{x \in C_a, y \in C_b} \|x - y\|.$$

Complete linkage:

$$d(C_a, C_b) = \max_{x \in C_a, y \in C_b} \|x - y\|.$$

Average linkage:

$$d(C_a, C_b) = \frac{1}{|C_a||C_b|} \sum_{x \in C_a} \sum_{y \in C_b} \|x - y\|.$$

Answer

Agglomeratives Clustering ist:

bottom-up: einzelne Punkte $\rightarrow$ große Cluster.

Die Wahl des Linkage-Kriteriums beeinflusst stark, welche Cluster entstehen.

8.10 Gaussian Mixture Models

Notes

Gaussian Mixture Models, kurz GMMs, modellieren Daten als Mischung mehrerer Gauß-Verteilungen.

Statt harter Clusterzuordnung wie bei k-means gibt es probabilistische Zugehörigkeiten zu Komponenten.

Solution

Ein GMM mit K Komponenten hat Parameter:

$$\pi_1, \dots, \pi_K$$

für die Mischgewichte,

$$\mu_1, \dots, \mu_K$$

für die Mittelwerte und

$$\Sigma_1, \dots, \Sigma_K$$

für die Kovarianzmatrizen.

Die Mischgewichte erfüllen:

$$\pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$

Calculation

Die Dichte eines GMMs lautet:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

Jede Komponente beschreibt einen möglichen Cluster.

Die Gesamtverteilung ist eine gewichtete Summe dieser Komponenten.

Answer

Ein Gaussian Mixture Model ist:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

Dabei sind π_k die Mischgewichte, μ_k die Mittelwerte und Σ_k die Kovarianzmatrizen.

8.11 Latent Variables and Responsibilities in GMMs

Notes

In GMMs kann man sich vorstellen, dass jeder Datenpunkt zuerst eine versteckte Clusterkomponente z_n auswählt.

Diese latente Variable ist nicht direkt beobachtet.

Sie gibt an, aus welcher Gauß-Komponente der Datenpunkt entstanden ist.

Solution

Für einen Datenpunkt x_n ist die posterior probability, dass er zu Komponente k gehört:

$$\gamma_{nk} = p(z_n = k \mid x_n).$$

Diese Größe heißt *responsibility*.

Sie beschreibt, wie stark Komponente k für Datenpunkt x_n verantwortlich ist.

Calculation

Nach Bayes' Regel gilt:

$$\gamma_{nk} = p(z_n = k \mid x_n) = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}.$$

Answer

Die Responsibility in einem GMM ist:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n | \mu_j, \Sigma_j)}$$

Sie ist eine weiche Clusterzuordnung.

8.12 Maximum Likelihood Parameter Estimation for GMMs

Notes

Die Parameter eines GMMs können mit Maximum Likelihood geschätzt werden.
Für Daten

$$x_1, \dots, x_N$$

lautet die Likelihood:

$$L(\theta) = \prod_{n=1}^N p(x_n | \theta).$$

Dabei enthält θ alle Mischgewichte, Mittelwerte und Kovarianzen.

Calculation

Für ein GMM ist:

$$p(x_n | \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k).$$

Damit lautet die Log-Likelihood:

$$\ell(\theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k) \right).$$

Das Problem ist, dass der Logarithmus über einer Summe steht.

Dadurch kann man die Parameter nicht so einfach direkt durch Ableiten lösen.

Answer

Die GMM-Log-Likelihood ist:

$$\ell(\theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k) \right)$$

Wegen des Logarithmus über der Summe verwendet man typischerweise den Expectation-Maximization-Algorithmus.

8.13 Free Energy and ELBO as Auxiliary Objectives

Notes

Die direkte Maximierung der GMM-Log-Likelihood ist schwierig.

Deshalb führt man ein Hilfsobjective ein: die Free Energy beziehungsweise Evidence Lower Bound, kurz ELBO.

Die Idee ist:

Optimiere eine untere Schranke der Log-Likelihood.

Solution

Für latente Variablen z gilt:

$$\log p(x) = \log \sum_z p(x, z).$$

Man führt eine Hilfsverteilung $q(z)$ ein.

Dann kann man schreiben:

$$\log p(x) = \log \sum_z q(z) \frac{p(x, z)}{q(z)}.$$

Mit Jensen's Ungleichung erhält man eine untere Schranke:

$$\log p(x) \geq \sum_z q(z) \log \frac{p(x, z)}{q(z)}.$$

Diese untere Schranke ist die ELBO.

Calculation

Für alle Daten lautet die ELBO:

$$\mathcal{L}(q, \theta) = \sum_{n=1}^N \sum_{k=1}^K q_{nk} \log \frac{\pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)}{q_{nk}}.$$

Dabei approximiert q_{nk} die Zugehörigkeit von Datenpunkt x_n zu Komponente k .

Im EM-Algorithmus wählt man im E-Schritt:

$$q_{nk} = \gamma_{nk} = p(z_n = k | x_n).$$

Answer

Die ELBO ist eine untere Schranke der Log-Likelihood:

$$\log p(x) \geq \sum_z q(z) \log \frac{p(x, z)}{q(z)}$$

Für GMMs führt diese Hilfsfunktion zum EM-Algorithmus:

E-Schritt: Verantwortlichkeiten berechnen

M-Schritt: Parameter mit diesen Verantwortlichkeiten aktualisieren.

8.14 Update of the Mean in GMMs

Notes

Im M-Schritt des EM-Algorithmus werden die Parameter des GMMs aktualisiert. Besonders wichtig ist das Update des Mittelwerts μ_k . Die Verantwortlichkeiten γ_{nk} wirken dabei wie weiche Gewichte.

Calculation

Definiere die effektive Anzahl von Punkten in Komponente k :

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Das Update des Mittelwerts lautet:

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n.$$

Das ist ein gewichteter Mittelwert der Datenpunkte.

Solution

Interpretation:

Wenn γ_{nk} groß ist, dann gehört x_n stark zu Komponente k und beeinflusst den Mittelwert μ_k stark.

Wenn γ_{nk} klein ist, dann hat x_n nur wenig Einfluss auf μ_k .

Im Unterschied zu k-means sind die Zuordnungen also nicht hart, sondern weich.

Answer

Das Mittelwert-Update im GMM ist:

$$\mu_k^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nk} x_n}{\sum_{n=1}^N \gamma_{nk}}$$

Es ist ein gewichteter Mittelwert mit Responsibilities γ_{nk} .

8.15 Relation Between k-means and GMMs

Notes

k-means und GMMs sind eng verwandt.

k-means verwendet harte Zuordnungen und Clusterzentren.

GMMs verwenden weiche Zuordnungen und probabilistische Gauß-Komponenten.

Solution

k-means kann als Grenzfall eines GMMs verstanden werden, wenn:

- alle Kovarianzmatrizen gleich und isotrop sind,
- die Varianz sehr klein wird,
- die weichen Responsibilities fast zu harten Zuordnungen werden.

Dann gehört jeder Punkt praktisch nur noch zur nächstgelegenen Komponente, ähnlich wie bei k-means.

Answer

Vergleich:

k-means: harte Zuordnungen, Zentren, quadratische Abstände

GMM: weiche Zuordnungen, Wahrscheinlichkeiten, Kovarianzen

GMMs sind flexibler, aber auch komplexer zu trainieren.

8.16 Discussion

Notes

Clustering ist grundsätzlich schwieriger zu bewerten als supervised learning, weil keine Labels gegeben sind.

Deshalb hängt die Qualität eines Clustering-Ergebnisses stark davon ab, welches Ziel man verfolgt.

Solution

Vorteile von k-means:

- einfach,
- schnell,
- gut für große Datensätze,
- leicht zu interpretieren.

Nachteile von k-means:

- Anzahl der Cluster K muss vorgegeben werden,
- empfindlich gegenüber Initialisierung,
- bevorzugt kugelförmige Cluster,
- harte Zuordnungen,
- empfindlich gegenüber Ausreißern.

Solution

Vorteile von hierarchischem Clustering:

- liefert eine ganze Hierarchie von Clustern,
- Anzahl der Cluster muss nicht sofort festgelegt werden,
- Dendrogramm ist interpretierbar.

Nachteile:

- oft rechenintensiver,
- frühe Zusammenführungen können nicht rückgängig gemacht werden,
- Ergebnis hängt stark vom Linkage-Kriterium ab.

Solution

Vorteile von GMMs:

- probabilistische Interpretation,
- weiche Clusterzuordnungen,
- unterschiedliche Clusterformen durch Kovarianzmatrizen,
- Unsicherheit der Zuordnung wird sichtbar.

Nachteile:

- komplexer als k-means,
- EM kann in lokalen Optima landen,
- Anzahl der Komponenten K muss gewählt werden,
- Kovarianzschätzung kann bei wenig Daten instabil sein.

Answer

Die Wahl des Clustering-Verfahrens hängt von der Datenstruktur ab:

k-means: einfache, kompakte Cluster

Hierarchisch: Clusterhierarchie und Dendrogramme

GMMs: probabilistische, weiche und elliptische Cluster

8.17 Prüfungsrelevante Kurzfassung

Answer

Lecture 8 behandelt Clustering mit k-means, hierarchischem Clustering und Gaussian Mixture Models.

Die wichtigsten Punkte sind:

- Clustering sucht Gruppen in ungelabelten Daten:

$$x_1, \dots, x_N.$$

- k-means minimiert:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2.$$

- k-means Zuordnungsupdate:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|x_n - \mu_j\|^2, \\ 0, & \text{sonst.} \end{cases}$$

- k-means Zentrenupdate:

$$\mu_k = \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}}.$$

- Fuzzy-k-means verwendet weiche Zuordnungen:

$$r_{nk} \in [0,1], \quad \sum_{k=1}^K r_{nk} = 1.$$

- Agglomeratives Clustering startet mit einzelnen Punkten und verbindet schrittweise die ähnlichsten Cluster.

- Ein Gaussian Mixture Model ist:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

- GMM-Responsibilities:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}.$$

- GMM-Log-Likelihood:

$$\ell(\theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k) \right).$$

- ELBO:

$$\log p(x) \geq \sum_z q(z) \log \frac{p(x, z)}{q(z)}.$$

- GMM-Mittelwertupdate:

$$\mu_k^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nk} x_n}{\sum_{n=1}^N \gamma_{nk}}.$$

Die Kernaussage lautet:

k-means, hierarchisches Clustering und GMMs sind verschiedene Wege, Gruppenstruktur in ungelabelten I

9 Lecture 9: Gaussian Mixture Models and the Expectation Maximization Algorithm

9.1 Overview

Notes

Lecture 9 setzt die Gaussian Mixture Models aus Lecture 8 fort und führt den Expectation-Maximization-Algorithmus, kurz EM-Algorithmus, systematisch ein.

Die zentralen Themen sind:

- Unterschied zwischen Log-Likelihood und Free Energy,
- Posterior-Verteilungen als optimale Wahl für die Hilfsverteilungen,
- iterative Berechnung optimaler Mittelwerte,
- ein erster EM-Algorithmus,
- Allgemeinheit des EM-Algorithmus,

- EM als Meta-Algorithmus für generative Modelle.

Answer

Die Kernaussage von Lecture 9 ist:

EM optimiert Modelle mit latenten Variablen, indem es abwechselnd Posterior-Verteilungen berechnet und

Für Gaussian Mixture Models bedeutet das: Erst berechnet man weiche Clusterzuordnungen, danach aktualisiert man die Modellparameter mit diesen Zuordnungen.

9.2 Gaussian Mixture Models Continued

Notes

Ein Gaussian Mixture Model beschreibt Daten als Mischung mehrerer Gauß-Verteilungen. Für K Komponenten lautet das Modell:

$$p(x) = \sum_{c=1}^K \pi_c \mathcal{N}(x \mid \mu_c, \Sigma_c).$$

Dabei sind:

- π_c die Mischgewichte,
- μ_c die Mittelwerte,
- Σ_c die Kovarianzmatrizen,
- c die latente Komponente beziehungsweise Clusterzugehörigkeit.

Solution

Die Variable c ist latent, weil sie nicht direkt beobachtet wird.

Für jeden Datenpunkt $x^{(n)}$ wissen wir nicht, aus welcher Komponente er erzeugt wurde.

Man kann sich den generativen Prozess so vorstellen:

1. Wähle eine Komponente:

$$c^{(n)} \sim \text{Categorical}(\pi_1, \dots, \pi_K).$$

2. Ziehe den Datenpunkt aus der gewählten Gauß-Verteilung:

$$x^{(n)} \sim \mathcal{N}(\mu_{c^{(n)}}, \Sigma_{c^{(n)}}).$$

Da $c^{(n)}$ unbekannt ist, müssen wir über alle möglichen Komponenten summieren.

Answer

Das GMM hat die Form:

$$p(x) = \sum_{c=1}^K \pi_c \mathcal{N}(x \mid \mu_c, \Sigma_c)$$

Die Clusterzugehörigkeit c ist eine latente Variable.

9.3 Difference Between Log-Likelihood and Free Energy

Notes

Für beobachtete Daten

$$x^{(1)}, \dots, x^{(N)}$$

lautet die Log-Likelihood eines GMMs:

$$\log p(X \mid \theta) = \sum_{n=1}^N \log p(x^{(n)} \mid \theta).$$

Wegen der latenten Variable c gilt:

$$p(x^{(n)} \mid \theta) = \sum_c p(x^{(n)}, c \mid \theta).$$

Also:

$$\log p(X \mid \theta) = \sum_{n=1}^N \log \sum_c p(x^{(n)}, c \mid \theta).$$

Solution

Das Problem ist der Ausdruck:

$$\log \sum_c p(x^{(n)}, c \mid \theta).$$

Der Logarithmus steht außerhalb der Summe über die latente Variable.

Dadurch ist die direkte Optimierung der Log-Likelihood schwierig, weil sich die Terme für die einzelnen Komponenten nicht einfach trennen lassen.

Die Free Energy beziehungsweise ELBO ersetzt dieses schwierige Objective durch eine untere Schranke, die leichter optimiert werden kann.

Calculation

Für jeden Datenpunkt führt man eine Hilfsverteilung

$$q^{(n)}(c)$$

über die latente Komponente ein.

Dann gilt:

$$\log p(x^{(n)} | \theta) = \log \sum_c p(x^{(n)}, c | \theta).$$

Da

$$\sum_c q^{(n)}(c) = 1,$$

kann man schreiben:

$$\log p(x^{(n)} | \theta) = \log \sum_c q^{(n)}(c) \frac{p(x^{(n)}, c | \theta)}{q^{(n)}(c)}.$$

Mit Jensen's Ungleichung erhält man:

$$\log p(x^{(n)} | \theta) \geq \sum_c q^{(n)}(c) \log \frac{p(x^{(n)}, c | \theta)}{q^{(n)}(c)}.$$

Calculation

Die Free Energy beziehungsweise ELBO für alle Daten ist:

$$\mathcal{F}(q, \theta) = \sum_{n=1}^N \sum_c q^{(n)}(c) \log \frac{p(x^{(n)}, c | \theta)}{q^{(n)}(c)}.$$

Damit gilt:

$$\log p(X | \theta) \geq \mathcal{F}(q, \theta).$$

Die Free Energy ist also eine untere Schranke an die Log-Likelihood.

Answer

Der Unterschied ist:

$$\log p(X | \theta) = \text{eigentliches Objective}$$

$$\mathcal{F}(q, \theta) = \text{untere Schranke / Hilfsobjective}$$

mit

$$\log p(X | \theta) \geq \mathcal{F}(q, \theta).$$

EM optimiert diese untere Schranke abwechselnd bezüglich q und θ .

9.4 Posterior Distributions as Optimal Choices for $q^{(n)}(c)$

Notes

Im E-Schritt des EM-Algorithmus wird die Hilfsverteilung

$$q^{(n)}(c)$$

optimal gewählt.

Die optimale Wahl ist der Posterior der latenten Variable:

$$q^{(n)}(c) = p(c \mid x^{(n)}, \theta).$$

Solution

Die Free Energy ist eine untere Schranke an die Log-Likelihood.

Der Abstand zwischen Log-Likelihood und Free Energy ist eine Kullback-Leibler-Divergenz:

$$\log p(X \mid \theta) - \mathcal{F}(q, \theta) = \sum_{n=1}^N \text{KL} \left(q^{(n)}(c) \parallel p(c \mid x^{(n)}, \theta) \right).$$

Da eine KL-Divergenz immer nichtnegativ ist, gilt:

$$\mathcal{F}(q, \theta) \leq \log p(X \mid \theta).$$

Die Schranke wird genau dann eng, wenn

$$q^{(n)}(c) = p(c \mid x^{(n)}, \theta)$$

für alle n .

Calculation

Für ein GMM lautet der Posterior:

$$p(c \mid x^{(n)}, \theta) = \frac{p(c \mid \theta) p(x^{(n)} \mid c, \theta)}{p(x^{(n)} \mid \theta)}.$$

Mit

$$p(c \mid \theta) = \pi_c$$

und

$$p(x^{(n)} \mid c, \theta) = \mathcal{N}(x^{(n)} \mid \mu_c, \Sigma_c)$$

folgt:

$$q^{(n)}(c) = \frac{\pi_c \mathcal{N}(x^{(n)} | \mu_c, \Sigma_c)}{\sum_{j=1}^K \pi_j \mathcal{N}(x^{(n)} | \mu_j, \Sigma_j)}.$$

Diese Größe nennt man auch Responsibility:

$$\gamma_{nc} = q^{(n)}(c).$$

Answer

Die optimale Wahl der Hilfsverteilung ist:

$$q^{(n)}(c) = p(c | x^{(n)}, \theta)$$

Für GMMs:

$$q^{(n)}(c) = \gamma_{nc} = \frac{\pi_c \mathcal{N}(x^{(n)} | \mu_c, \Sigma_c)}{\sum_{j=1}^K \pi_j \mathcal{N}(x^{(n)} | \mu_j, \Sigma_j)}$$

Damit wird die Free-Energy-Schranke an der aktuellen Stelle eng.

9.5 Computation of Optimal Means Using an Iterative Algorithm

Notes

Wenn die Zuordnungen der Datenpunkte zu den Komponenten unbekannt sind, können die Mittelwerte μ_c nicht direkt wie bei vollständig gelabelten Daten berechnet werden.

EM löst dieses Problem iterativ:

- Berechne weiche Zuordnungen $q^{(n)}(c)$.
- Aktualisiere Mittelwerte mit diesen weichen Zuordnungen.
- Wiederhole.

Calculation

Für fixe Responsibilities $q^{(n)}(c)$ enthält die Free Energy bezüglich der Mittelwerte Terme der Form:

$$\sum_{n=1}^N \sum_c q^{(n)}(c) \log \mathcal{N}(x^{(n)} | \mu_c, \Sigma_c).$$

Für eine Gauß-Verteilung ist der relevante Teil:

$$-\frac{1}{2} \sum_{n=1}^N q^{(n)}(c) (x^{(n)} - \mu_c)^T \Sigma_c^{-1} (x^{(n)} - \mu_c).$$

Maximierung bezüglich μ_c ergibt einen gewichteten Mittelwert.

Calculation

Definiere:

$$N_c = \sum_{n=1}^N q^{(n)}(c).$$

Dann lautet das Update:

$$\mu_c^{\text{new}} = \frac{1}{N_c} \sum_{n=1}^N q^{(n)}(c) x^{(n)}.$$

Mit Responsibilities γ_{nc} schreibt man:

$$\mu_c^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nc} x^{(n)}}{\sum_{n=1}^N \gamma_{nc}}.$$

Answer

Das iterative Mittelwertupdate ist:

$$\mu_c^{\text{new}} = \frac{\sum_{n=1}^N q^{(n)}(c) x^{(n)}}{\sum_{n=1}^N q^{(n)}(c)}$$

beziehungsweise:

$$\mu_c^{\text{new}} = \frac{\sum_{n=1}^N \gamma_{nc} x^{(n)}}{\sum_{n=1}^N \gamma_{nc}}.$$

Das ist ein gewichteter Mittelwert der Datenpunkte.

9.6 A First EM Algorithm for GMM Means

Notes

Ein erster einfacher EM-Algorithmus für GMMs kann formuliert werden, wenn man sich zunächst auf die Aktualisierung der Mittelwerte konzentriert.

Die anderen Parameter, zum Beispiel π_c und Σ_c , können dabei zunächst als fix angenommen werden.

Solution

Der Algorithmus läuft iterativ:

1. Initialisiere die Mittelwerte μ_c .
2. Berechne für jeden Datenpunkt und jede Komponente die Responsibilities:

$$\gamma_{nc} = p(c \mid x^{(n)}, \theta).$$

3. Aktualisiere die Mittelwerte:

$$\mu_c \leftarrow \frac{\sum_{n=1}^N \gamma_{nc} x^{(n)}}{\sum_{n=1}^N \gamma_{nc}}.$$

4. Wiederhole E-Schritt und M-Schritt bis zur Konvergenz.

Answer

Ein erster EM-Algorithmus:

$$\text{E-Schritt: } \gamma_{nc} = p(c \mid x^{(n)}, \theta)$$

$$\text{M-Schritt: } \mu_c \leftarrow \frac{\sum_{n=1}^N \gamma_{nc} x^{(n)}}{\sum_{n=1}^N \gamma_{nc}}$$

EM wechselt also zwischen Schätzen der latenten Zuordnung und Aktualisieren der Parameter.

9.7 The Generality of the EM Algorithm

Notes

Der EM-Algorithmus ist nicht auf Gaussian Mixture Models beschränkt.

Er ist ein allgemeines Verfahren für Modelle mit latenten Variablen.

Die Struktur ist:

beobachtete Daten X , latente Variablen Z , Parameter θ .

Solution

In vielen Modellen wäre die Maximum-Likelihood-Schätzung einfach, wenn die latenten Variablen bekannt wären.

Da sie aber unbekannt sind, arbeitet EM mit Posterior-Verteilungen über die latenten Variablen.

Der E-Schritt berechnet die aktuelle Unsicherheit über Z .

Der M-Schritt aktualisiert die Parameter so, als würde man diese weichen Zuordnungen als Gewichte verwenden.

Answer

EM ist allgemein ein Algorithmus für Modelle der Form:

$$p(x, z \mid \theta)$$

wobei x beobachtet und z latent ist.

Ziel ist die Maximierung der marginalen Likelihood:

$$p(x \mid \theta) = \sum_z p(x, z \mid \theta)$$

beziehungsweise im kontinuierlichen Fall:

$$p(x \mid \theta) = \int p(x, z \mid \theta) dz.$$

9.8 Derivations Do Not Depend on Specific Distribution Forms

Notes

Eine wichtige Eigenschaft von EM ist, dass die grundlegende Herleitung nicht von der konkreten Form der Verteilungen abhängt.

Die Herleitung verwendet nur:

- latente Variablen,
- Marginalisierung,
- Jensen's Ungleichung,
- Posterior-Verteilungen,
- Maximierung eines Hilfsobjectives.

Solution

Für GMMs sind die latenten Variablen diskrete Clusterindikatoren.

In anderen Modellen können die latenten Variablen aber anders aussehen:

- diskrete Klassen,
- kontinuierliche latente Variablen,
- fehlende Daten,
- Sequenzen verborgener Zustände,
- Mischkomponenten,
- versteckte Ursachen.

Die konkrete Form der E- und M-Schritte hängt vom Modell ab.

Das Prinzip bleibt aber gleich.

Answer

Die EM-Herleitung ist allgemein, weil sie auf der Struktur

$$\log p(x | \theta) = \log \sum_z p(x, z | \theta)$$

beziehungsweise

$$\log p(x | \theta) = \log \int p(x, z | \theta) dz$$

basiert.

Sie hängt nicht speziell davon ab, dass die Komponenten Gauß-Verteilungen sind.

9.9 Forms for Different Latent Variables

Notes

Je nach Modell kann die latente Variable verschiedene Formen haben.

Die EM-Idee bleibt dieselbe, aber Summen und Integrale ändern sich abhängig vom Typ der latenten Variable.

Solution

Für diskrete latente Variablen verwendet man Summen:

$$p(x | \theta) = \sum_z p(x, z | \theta).$$

Für kontinuierliche latente Variablen verwendet man Integrale:

$$p(x | \theta) = \int p(x, z | \theta) dz.$$

Für mehrere latente Variablen können entsprechend mehrfache Summen oder Integrale auftreten.

Answer

Die Form hängt vom Latenttyp ab:

$$\text{diskretes } z : \quad p(x | \theta) = \sum_z p(x, z | \theta)$$

$$\text{kontinuierliches } z : \quad p(x | \theta) = \int p(x, z | \theta) dz$$

EM bleibt aber strukturell gleich.

9.10 Definition of the EM Meta-Algorithm for Generative Models

Notes

Der EM-Algorithmus kann als Meta-Algorithmus für generative Modelle mit latenten Variablen formuliert werden.

Gegeben ist ein Modell:

$$p(x, z \mid \theta).$$

Beobachtet wird nur x , nicht aber z .

Calculation

Die marginale Log-Likelihood lautet:

$$\log p(X \mid \theta) = \sum_{n=1}^N \log \sum_z p(x^{(n)}, z \mid \theta).$$

EM maximiert stattdessen iterativ die Free Energy:

$$\mathcal{F}(q, \theta) = \sum_{n=1}^N \sum_z q^{(n)}(z) \log \frac{p(x^{(n)}, z \mid \theta)}{q^{(n)}(z)}.$$

Solution

Der EM-Meta-Algorithmus:

1. Initialisiere die Parameter θ^{old} .
2. **E-Schritt:** Berechne die Posterior-Verteilung der latenten Variablen unter den aktuellen Parametern:

$$q^{(n)}(z) = p(z \mid x^{(n)}, \theta^{old}).$$

3. **M-Schritt:** Aktualisiere die Parameter durch Maximierung der Free Energy:

$$\theta^{new} = \arg \max_{\theta} \mathcal{F}(q, \theta).$$

4. Wiederhole E- und M-Schritt bis zur Konvergenz.

Answer

Der EM-Meta-Algorithmus lautet:

$$\text{E-Schritt: } q^{(n)}(z) = p(z \mid x^{(n)}, \theta^{old})$$

$$\text{M-Schritt: } \theta^{new} = \arg \max_{\theta} \mathcal{F}(q, \theta)$$

mit

$$\mathcal{F}(q, \theta) = \sum_n \sum_z q^{(n)}(z) \log \frac{p(x^{(n)}, z | \theta)}{q^{(n)}(z)}.$$

9.11 Prüfungsrelevante Kurzfassung

Answer

Lecture 9 behandelt GMMs und die allgemeine Idee des Expectation-Maximization-Algorithmus.

Die wichtigsten Punkte sind:

- Ein GMM hat die Form:

$$p(x) = \sum_{c=1}^K \pi_c \mathcal{N}(x | \mu_c, \Sigma_c).$$

- Die Log-Likelihood enthält einen schwierigen Term:

$$\log \sum_c p(x^{(n)}, c | \theta).$$

- Die Free Energy ist eine untere Schranke:

$$\log p(X | \theta) \geq \mathcal{F}(q, \theta).$$

- Die Free Energy lautet:

$$\mathcal{F}(q, \theta) = \sum_n \sum_c q^{(n)}(c) \log \frac{p(x^{(n)}, c | \theta)}{q^{(n)}(c)}.$$

- Die optimale Wahl von q ist der Posterior:

$$q^{(n)}(c) = p(c | x^{(n)}, \theta).$$

- Für GMMs sind die Responsibilities:

$$\gamma_{nc} = \frac{\pi_c \mathcal{N}(x^{(n)} | \mu_c, \Sigma_c)}{\sum_j \pi_j \mathcal{N}(x^{(n)} | \mu_j, \Sigma_j)}.$$

- Das Mittelwertupdate ist:

$$\mu_c^{new} = \frac{\sum_n \gamma_{nc} x^{(n)}}{\sum_n \gamma_{nc}}.$$

- EM ist allgemein für Modelle mit latenten Variablen:

$$p(x, z \mid \theta).$$

- E-Schritt:

$$q^{(n)}(z) = p(z \mid x^{(n)}, \theta^{old}).$$

- M-Schritt:

$$\theta^{new} = \arg \max_{\theta} \mathcal{F}(q, \theta).$$

Die Kernaussage lautet:

EM optimiert Modelle mit latenten Variablen durch abwechselnde Posterior-Berechnung und Parameteraktualisierung.